

An efficient implementation of nodal discontinuous Galerkin lattice Boltzmann method and validation for direct numerical simulation of turbulent flows

Boyang Xia^a, Jun Li^{a,b,*}

^a Institute of Turbomachinery, Xi'an Jiaotong University, Xi'an 710049, PR China

^b Collaborative Innovation Centre of Advanced Aero-Engine, Beijing 100191, PR China

ARTICLE INFO

Keywords:

Nodal discontinuous Galerkin method
Lattice Boltzmann method
High performance computing
Direct numerical simulation

ABSTRACT

The application of finite element method in solving the lattice Boltzmann equation has been successfully demonstrated by many researchers. However, the computation cost of these methods is much higher than the standard lattice Boltzmann method. In this paper, an efficient implementation of nodal discontinuous Galerkin lattice Boltzmann method is presented. It relies on a simplified nodal discontinuous Galerkin formulation, which makes full use of the possible sparse property of elemental matrices in quadrangles and hexahedrons. Some key points of its implementation in the program are also discussed. Performance tests confirm the superior efficiency of the current program, which exceeds the record of previous researchers. The accuracy of the current method is investigated and compared with the standard lattice Boltzmann method. The high-order discretization can give similar accuracy with lower number of grid points and computational time and the efficiency of the present method is sometimes higher than standard lattice Boltzmann method. Direct numerical simulations of several incompressible turbulence benchmark cases are performed. The present method also shows better accuracy and stability especially in the turbulent pipe flow simulations.

1. Introduction

The lattice Boltzmann method (LBM) is a promising alternative to conventional computational fluid dynamics methods based on Navier-Stokes equations and has been researched and applied widely in past decades. It emerges out of kinetic theory with decoupled streaming and collision processes and regular grid discretization, which is suitable for problems with complex geometries such as multi-phase flow and porous media flow. Moreover, in the computing aspect, the collision step is purely local and the on-lattice streaming step can be done via memory shift, which is free of floating-point operations. Therefore, LBM is famous for its excellent computing performance.

The inherent features of LBM also bring side effects. First, regular grid discretization achieves staircase approximation near curved boundaries, which produces lower than second-order accuracy. Second, lattice discretization may produce checkerboard instability with smaller velocity sets like D3Q19 and D3Q15 [1], especially when symmetrical property and boundary accuracy is emphasized. One typical example is fully developed turbulent pipe flow where the D3Q19 is reported to be

hard to obtain correct results with coarse grids [2,3]. Finally, the fixed relation between the time step size and the lattice size restricts the free tuning of parameters, making it effectively first-order accuracy in time [4].

To overcome these shortcomings, many researchers have turned to other discretization approaches such as finite difference (FD) [6], finite volume (FV) [7], finite element (FE) [8,10–13], and interpolation-based method [5]. The discontinuous Galerkin (DG) method combines the merits of finite element and finite volume methods, which can easily obtain high-order spatial accuracy on complex geometries via spectral element and still preserve a semi-discrete scheme similar to FV for efficient time marching. It has been widely applied in solving hyperbolic systems [9], of which the Boltzmann equation is also a typical example. Shi et al. [10] and Düster et al. [11] used DG to solve discrete velocity Boltzmann equation (DVBE) directly. However, stiffness problem occurs in the collision term when relaxation time is close to zero. Implicit time marching can deal with this issue but comes with high computing costs. Min and Lee [12] provide a better option. They solve only the streaming step and retain the original collision step, which avoids the stiffness is-

* Corresponding author at: Institute of Turbomachinery, Xi'an Jiaotong University, Xi'an 710049, PR China.

E-mail addresses: xiaoyang@stu.xjtu.edu.cn (B. Xia), junli@mail.xjtu.edu.cn (J. Li).

sue and allows larger time step sizes for explicit time marching. Besides the above experiences, other ways of combining the high-order finite element with the lattice Boltzmann method also exist. One interesting example is provided by Karakus et al. [14] which discretizes Boltzmann equations in a novel way with fewer equations to be solved by DG. Another example, which is becoming more popular recently, is to solve streaming step by the semi-Lagrangian scheme, allowing significantly larger time step sizes and giving better efficiency [15]. Recently, the semi-Lagrangian lattice Boltzmann method (SLLBM) has been successfully extended to compressible flow [16], indicating a promising future of these methods.

Nevertheless, the computing cost of DGLBM and other high-order FE-based lattice Boltzmann methods is a real concern compared with standard LBM. The streaming equation associated with each discrete velocity needs to be solved via DG discretization. For 3-dimensional weakly compressible flow, 13~27 discrete velocities are usually adequate while for the compressible flow much more velocities are required [17]. This causes high computing cost. It is reported that over 90% of the runtime is consumed by DG streaming step [18]. Therefore, an efficient implementation of the DG method is vital to reduce the computing cost and improve the competitiveness of DGLBM.

In the past few decades, the implementation techniques of the DG method with high-order spectral elements have been thoroughly studied by many researchers [9,19]. For simplex elements with affine mapping, the quadrature-free implementation given by Atkins et al. [20] precomputes and stores the inner product of orthogonal bases and shares it to every element to eliminate explicit quadrature during computing. Such idea has been extended to nodal DG method by Hesthaven and Warburton [9]. For quadrangles and hexahedrons, explicit numerical quadrature is necessary, but the tensor-product expansions provide a chance to reduce the computing cost by sum-factorization, initially proposed for spectral methods. In fact, many techniques originally from spectral method such as even-odd decomposition and sum-factorization have been adopted in the open-source finite element library like deal.II [24] to speed up computation. In the present work, besides taking advantage of the tensor-product property in the implementation of DG, we provide some other simplifications specially designed for linear advection equations of DGLBM.

Several researchers have numerically validated DGLBM but so far only Krämer et al. [18] provided a convergence study on order of accuracy. The present work gives a more detailed convergence study in which temporal, spatial and modeling errors are revealed clearly. These tests also provide guidelines for discretization parameters and time marching method. Moreover, the effect of h/p refinement on direct numerical simulation (DNS) of turbulence is analyzed via the three-dimensional Taylor-Green vortex. And for the first time, the fully developed turbulent pipe flow is simulated by DGLBM to demonstrate its capability in wall-bounded turbulence simulations.

The present work is organized as follows. In section 2, the basic formulation of DGLBM is briefly introduced and a simplified nodal DG formulation is presented and discussed in detail. In section 3, we focus on the efficient implementation of the DGLBM in the parallel program and provides the performance tests and comparison. In section 4, a convergence study is given followed by direct numerical simulations of several turbulence benchmark cases. Concluding remarks are given at the end of the paper.

2. Formulation

2.1. Discontinuous Galerkin lattice Boltzmann method

The lattice Boltzmann equation (LBE) with Bhatnagar–Gross–Krook (BGK) collision operator and second-order force scheme reads [4]:

$$\tilde{f}_\alpha(\mathbf{x} + \mathbf{e}_\alpha \Delta t, t + \Delta t) - \tilde{f}_\alpha(\mathbf{x}, t) = sv(f_\alpha^{eq} - \tilde{f}_\alpha) + \left(1 - \frac{1}{2}sv\right) \Delta t \hat{F}_\alpha, \quad (1)$$

where $\tilde{f}_\alpha$ denotes the modified particle distribution function [12] and $sv = (\tau_f/\Delta t + 1/2)^{-1}$ is the dimensionless relaxation factor which relates to the fluid kinematic viscosity ν and sound speed c_s by $\tau_f = \nu/c_s^2$. For isothermal weakly compressible fluid flow, the following second order truncation of equilibrium distribution function is used:

$$f_\alpha^{eq} = \rho w_\alpha \left[1 + \frac{\mathbf{e}_\alpha \cdot \mathbf{u}}{c_s^2} + \frac{(\mathbf{e}_\alpha \cdot \mathbf{u})^2}{2c_s^4} - \frac{\mathbf{u}^2}{2c_s^2} \right]. \quad (2)$$

w_α is the weight associated with each discrete velocity $\mathbf{e}_\alpha$ and $\mathbf{u}$ is the macroscopic flow velocity. Throughout this paper, the well-known D2Q9 and D3Q19 lattices [4] are adopted for two-dimensional and three-dimensional simulations, respectively. $\hat{F}_\alpha$ is the forcing term applied to each discrete velocity. In present work, we use the forcing term given by Guo et al. [21]:

$$\hat{F}_\alpha = w_\alpha \left[\frac{e_{\alpha j}}{c_s^2} + \frac{(e_{\alpha i} e_{\alpha j} - c_s^2 \delta_{ij}) u_i}{c_s^4} \right] F_j, \quad (3)$$

where F_j is the body force vector. With this forcing scheme, the macroscopic quantities are given as follows:

$$\rho = \sum_\alpha \tilde{f}_\alpha + \frac{\Delta t}{2} \sum_\alpha \hat{F}_\alpha, \quad (4)$$

$$\rho \mathbf{u} = \sum_\alpha \tilde{f}_\alpha \mathbf{e}_\alpha + \frac{\Delta t}{2} \sum_\alpha \hat{F}_\alpha \mathbf{e}_\alpha. \quad (5)$$

The LBE is commonly solved via two steps, the collision step:

$$\tilde{f}_\alpha^* = \tilde{f}_\alpha + sv(f_\alpha^{eq} - \tilde{f}_\alpha) + \left(1 - \frac{1}{2}sv\right) \Delta t \hat{F}_\alpha, \quad (6)$$

and the streaming step:

$$\tilde{f}_\alpha(\mathbf{x} + \mathbf{e}_\alpha \Delta t, t + \Delta t) = \tilde{f}_\alpha^*(\mathbf{x}, t). \quad (7)$$

In DGLBM given by Min et al. [12], collision step (6) is preserved while the streaming step (7) is replaced by solving its equivalent equation in Eulerian framework:

$$\frac{\partial \tilde{f}_\alpha}{\partial t} + \mathbf{e}_\alpha \cdot \nabla \tilde{f}_\alpha = 0. \quad (8)$$

The above form of linear advection equation has been widely used as an example to illustrate the DG discretization procedures. For brevity, we directly give the strong formulation [9] as follows:

$$\int_{\Omega^k} \left(\frac{\partial \tilde{f}_\alpha}{\partial t} + \nabla \cdot \mathbf{F}(\tilde{f}_\alpha) \right) \phi dx = \oint_{\partial \Omega^k} \mathbf{n} \cdot (\mathbf{F} - \mathbf{F}^*) \phi d\mathbf{x}, \quad (9)$$

where $\mathbf{F}$ represents flux term $\mathbf{e}_\alpha \tilde{f}_\alpha$ and $\mathbf{F}^*$ is the numerical flux. $\partial \Omega^k$ denotes the boundary of element domain Ω^k and $\mathbf{n}$ is its unit normal vector. In the nodal DG method, the test function ϕ is the same as nodal expansion basis ℓ_i . The solution is expressed by ℓ_i as follows:

$$\tilde{f}_\alpha(\mathbf{x}, t) \cong \sum_{i=1}^{N_p} \tilde{f}_\alpha(\mathbf{x}_i, t) \ell_i(\mathbf{x}). \quad (10)$$

Here $\mathbf{x}_i$ denotes the coordinate of the nodal point and N_p is number of nodal points in one element. $\tilde{f}_\alpha(\mathbf{x}_i, t)$ is the unknown at points $\mathbf{x}_i$ which forms the solution vector $\tilde{\mathbf{f}}_\alpha = \tilde{f}_{\alpha,i}$. In practice, the expansion basis is defined on reference element I for sake of convenience and the mapping function is defined to transform the global coordinates $\mathbf{x} = (x, y, z)$ of the physical element Ω^k into local coordinates $\mathbf{r} = (r, s, t)$ in the reference element I . With these definitions, Eq. (9) can be transformed into matrix notation:

$$\frac{d\tilde{\mathbf{f}}_{\alpha,i}}{dt} = -(M)_{ik}^{-1} S_{\alpha,kj} \tilde{f}_{\alpha,j} + \sum_{\text{edge}} (M)_{ij}^{-1} B_{\alpha,j} \quad (11)$$

and the element mass matrix M_{ij} , stiffness matrix $S_{\alpha,ij}$, and boundary flux contribution $B_{\alpha,i}$ from each edge of element are

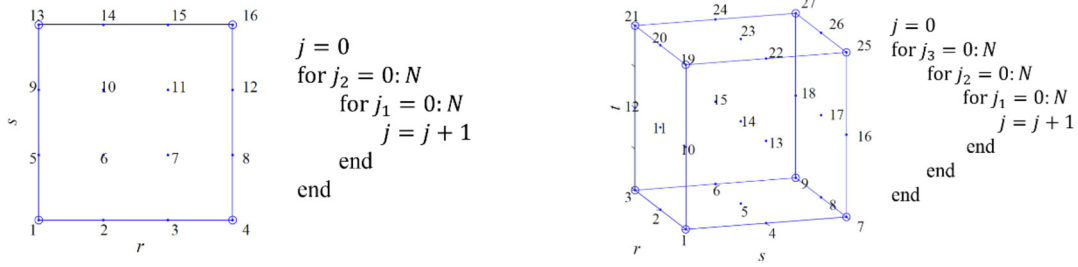


Fig. 1. An example of nodal points indexing for quadrilateral (left) and hexahedral (right) elements.

$$M_{ij} = \int_I \ell_i(\mathbf{r}) \ell_j(\mathbf{r}) J^c d\mathbf{r}, \quad (12)$$

$$S_{\alpha,ij} = \int_I \ell_i(\mathbf{r}) \frac{\partial \ell_j(\mathbf{r})}{\partial \mathbf{r}} \frac{\partial \mathbf{r}}{\partial \mathbf{x}} \cdot \mathbf{e}_\alpha J^c d\mathbf{r}, \quad (13)$$

$$B_{\alpha,i} = \int_{\text{edge}} \ell_i(\mathbf{r}) (\mathbf{F} - \mathbf{F}^*) \cdot \mathbf{n} J^e d\mathbf{r}, \quad (14)$$

where J^c and J^e are the coordinate mapping Jacobians of cell and edge, respectively. Assuming that the boundary flux $\mathbf{n} \cdot (\mathbf{F} - \mathbf{F}^*)$ can be represented by Lagrangian polynomials of the same order N , then $B_{\alpha,i}$ can be expressed as an edge mass matrix M_{ij}^e which is independent of unknowns, multiplied by boundary flux $g_j = [\mathbf{n} \cdot (\mathbf{F} - \mathbf{F}^*)]_{r_j}$ related to unknowns:

$$B_{\alpha,i} = g_j \int_{\text{edge}} \ell_i(\mathbf{r}) \ell_j(\mathbf{r}) J^e d\mathbf{r} = M_{ij}^e g_j. \quad (15)$$

The right hand side of semi-discrete form Eq. (11) now can be evaluated via two matrix-vector multiplication:

$$\frac{d\tilde{f}_{\alpha,i}}{dt} = -M_{ij}^{\text{diff}} \tilde{f}_{\alpha,j} + \sum_{\text{edge}} M_{ij}^{\text{lift}} g_j. \quad (16)$$

This serves as the baseline of nodal DG formulation for DGLBM, where $M_{ij}^{\text{diff}} = (M)_{ik}^{-1} S_{\alpha,kj}$ and $M_{ij}^{\text{lift}} = (M)_{ik}^{-1} M_{kj}^e$. Later we will see that it is possible to construct sparse M_{ij}^{diff} and M_{ij}^{lift} to reduce the computing cost further.

The mass matrix M_{ij} is purely local for one element and can be inverted easily. So Eq. (11) has been written in a semi-discrete form, which can be solved later by explicit time marching efficiently. To reduce the memory cost and improve performance, low-storage explicit Runge-Kutta (RK) methods are favored. A typical form is given as follows:

$$\begin{aligned} \mathbf{f}^0 &= \tilde{\mathbf{f}}_\alpha(t) \\ d\mathbf{f}^k &= a_k d\mathbf{f}^{k-1} + \Delta t L_h(\mathbf{f}^{k-1}), \mathbf{f}^k = \mathbf{f}^{k-1} + b_k d\mathbf{f}^k, k = 1, 2, \dots, s \\ \tilde{\mathbf{f}}_\alpha(t + \Delta t) &= \mathbf{f}^s \end{aligned} \quad (17)$$

where s is the stage number and a_k, b_k are coefficients. A class of optimal coefficients given by Ruuth and Spiteri [22] are adopted here and called Strong-Stability-Preserving and Low-storage Runge-Kutta (SS-PLRK) method. The widely-used five-stage fourth-order low-storage RK method given by Carpenter and Kennedy [23] is also tested.

2.2. A simplified nodal discontinuous Galerkin formulation

2.2.1. Simplification of upwind Lax-Friedrichs flux

Preliminary study by Min et al. [12] has proved the stability of upwind Lax-Friedrichs flux in solving Eq. (11), which is

$$\mathbf{F}^* = \frac{1}{2} [\mathbf{F}(\tilde{f}_\alpha) + \mathbf{F}(\tilde{f}_\alpha^+) + |\Lambda|(\tilde{f}_\alpha - \tilde{f}_\alpha^+) \mathbf{n}], \quad (18)$$

here, $\Lambda = \max(\mathbf{n} \cdot \frac{\partial \mathbf{F}}{\partial \mathbf{f}_\alpha}) = \mathbf{n} \cdot \mathbf{e}_\alpha$ and then boundary flux g_j can be simplified as

$$g_j = \frac{1}{2} (\mathbf{n} \cdot \mathbf{e}_\alpha - |\mathbf{n} \cdot \mathbf{e}_\alpha|) (\tilde{f}_\alpha - \tilde{f}_\alpha^+) = \begin{cases} \mathbf{n} \cdot \mathbf{e}_\alpha (\tilde{f}_\alpha - \tilde{f}_\alpha^+), & \mathbf{n} \cdot \mathbf{e}_\alpha < 0 \\ 0, & \mathbf{n} \cdot \mathbf{e}_\alpha \geq 0 \end{cases} \quad (19)$$

which indicates in half of the equations (denoted by α) we do not need to evaluate the boundary flux term. This simplification is valid as long as the advection flux term is linear and $\frac{\partial \mathbf{F}}{\partial \mathbf{f}_\alpha}$ can be constant.

2.2.2. Sparse differentiation matrix in tensor-product element

If the derivative of Lagrangian polynomial basis in Eq. (13) can also be represented by N -th-order Lagrangian polynomials, namely,

$$\frac{\partial \ell_j(\mathbf{r})}{\partial \mathbf{r}} \frac{\partial \mathbf{r}}{\partial \mathbf{x}} \cong \sum_{n=1}^{N_p} \ell_n(\mathbf{r}) \left(\frac{\partial \ell_j}{\partial \mathbf{r}} \frac{\partial \mathbf{r}}{\partial \mathbf{x}} \right)_{r_n}, \quad (20)$$

then the stiffness matrix can be transformed into

$$\begin{aligned} S_{\alpha,ij} &= \sum_{n=1}^{N_p} \mathbf{e}_\alpha \cdot \left(\frac{\partial \ell_j}{\partial \mathbf{r}} \frac{\partial \mathbf{r}}{\partial \mathbf{x}} \right)_{r_n} \int_I \ell_i(\mathbf{r}) \ell_n(\mathbf{r}) J^c d\mathbf{r} \\ &= M_{in} \{ [\text{diag}(r_x) D_{r,nj} + \text{diag}(s_x) D_{s,nj}] e_{\alpha x} + [\text{diag}(r_y) D_{r,nj} \\ &\quad + \text{diag}(s_y) D_{s,nj}] e_{\alpha y} \} \end{aligned} \quad (21)$$

Here, we take two-dimensional case as an example. $D_{r,nj} = (\frac{d\ell_j}{dr})_{r_n}$ and $D_{s,nj} = (\frac{d\ell_j}{ds})_{r_n}$ are differentiation matrices [9] defined as the derivative of the j -th Lagrangian polynomial basis at nodal point n . $\text{diag}(r_x)$ is the diagonal matrix consisting of $\frac{\partial \mathbf{r}}{\partial x}$ at each nodal point r_n . In the final formula of M_{ij}^{diff} , the mass matrix M_{in} is canceled by multiplying its inverse so we get

$$\begin{aligned} M_{ij}^{\text{diff}} &= D_{x,ij} e_{\alpha x} + D_{y,ij} e_{\alpha y} \\ &= [\text{diag}(r_x) D_{r,ij} + \text{diag}(s_x) D_{s,ij}] e_{\alpha x} \\ &\quad + [\text{diag}(r_y) D_{r,ij} + \text{diag}(s_y) D_{s,ij}] e_{\alpha y}. \end{aligned} \quad (22)$$

It is clear that sparse property of M_{ij}^{diff} solely depends on differentiation matrices $D_{r,nj}$ and $D_{s,nj}$. And luckily it is possible to construct sparse differentiation matrices in tensor-product elements, namely, quadrilateral and hexahedral elements.

The nodal expansion of quadrilateral element can be constructed by tensor-product of one-dimensional element:

$$\ell_j(r, s) = \ell_{j_1}^{1d}(r) \ell_{j_2}^{1d}(s), \quad (23)$$

where the superscript $1d$ is used to distinguish the one-dimensional case from the multi-dimensional case. The indices of the nodal points j and (j_1, j_2) are given in Fig. 1. Considering that $\ell_j^{1d}(r_i) = \delta_{ij}$, we can get the simplified differentiation matrices as follows.

$$\begin{aligned} D_{r,ij} &= \begin{cases} \frac{d\ell_{j_1}^{1d}}{dr}(r_{i_1}) = D_{r,i_1j_1}^{1d}, & i_2 = j_2; \\ 0, & i_2 \neq j_2 \end{cases} \\ D_{s,ij} &= \begin{cases} \frac{d\ell_{j_2}^{1d}}{ds}(s_{i_2}) = D_{s,i_2j_2}^{1d}, & i_1 = j_1; \\ 0, & i_1 \neq j_1, \end{cases} \end{aligned} \quad (24)$$

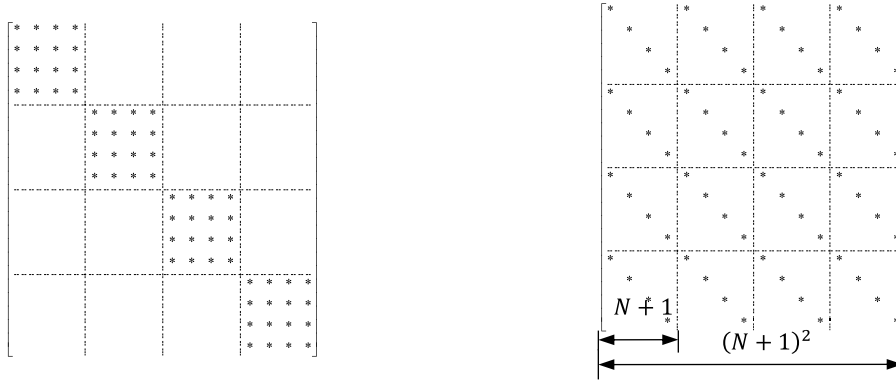


Fig. 2. Sparse pattern of differentiation matrices $D_{r,ij}$ (left) and $D_{s,ij}$ (right) in 3rd order quadrilateral element.

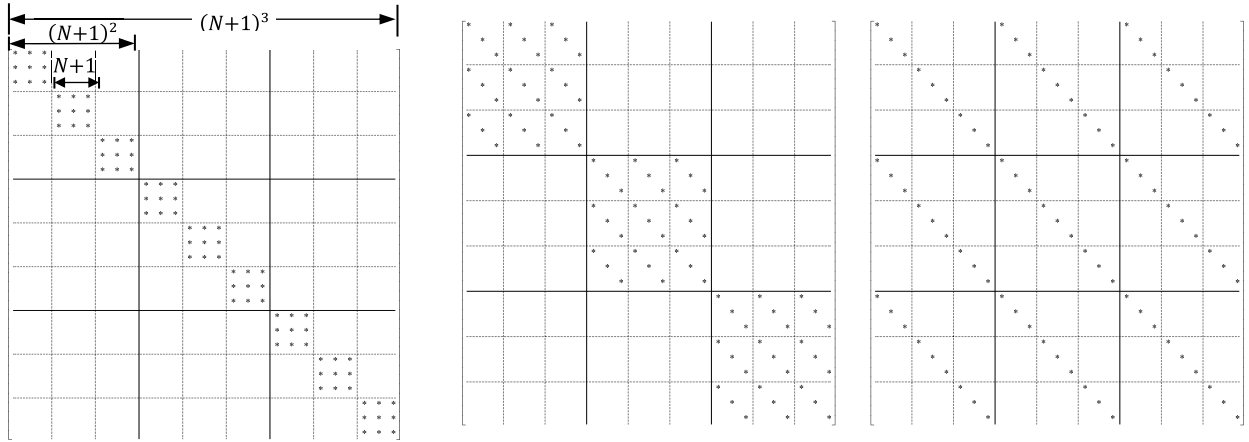


Fig. 3. Sparse pattern of differentiation matrices $D_{r,ij}$ (left), $D_{s,ij}$ (middle), $D_{t,ij}$ (right) in 2nd order hexahedral element.

where i and (i_1, i_2) are also indices of the nodal points.

As shown in Fig. 2, Eq. (24) result in very sparse $D_{r,ij}$ and $D_{s,ij}$ with only $(N+1)$ non-zeros for each row or column. The final M_{ij}^{diff} which is the superposition of $D_{r,ij}$ and $D_{s,ij}$ is still sparse enough with $(dN+1)$ non-zeros for each row or column. The differentiation matrices in hexahedral elements also have similar results, as shown in Fig. 3. The above derivation does not involve any specific nodal basis. So the result is true for all nodal quadrilateral and hexahedral elements.

2.2.3. The same points for both nodal basis and quadrature

For general type of element, the evaluation of mass matrix requires explicit numerical quadrature which reads

$$M_{ij} = \sum_{n=1}^{N_c} \ell_i(\mathbf{r}_n^c) \ell_j(\mathbf{r}_n^c) J_n^c w_n^c, M_{ij}^e = \sum_{m=1}^{N_e} \ell_i(\mathbf{r}_m^e) \ell_j(\mathbf{r}_m^e) J_m^e w_m^e \quad (25)$$

where the superscripts c and e are used to distinguish the cell quadrature and edge quadrature. $\mathbf{r}_n^c$ and $\mathbf{r}_m^e$ are quadrature points. w_n^c and w_m^e are corresponding weights. J_n^c and J_m^e are coordinate mapping Jacobians at quadrature points.

A well-known simplification is to use N -th-order Gauss-Lobatto points for both nodal basis and quadrature. Then $\ell_i(\mathbf{r}_n^c) = \delta_{in}$ and element mass matrix becomes diagonal: $\mathbf{M} = \text{diag}(J_n^c w_n^c)$. Furthermore, the Gauss-Lobatto points can also be used for edge quadrature. Then $\ell_i(\mathbf{r}_m^e) \neq 0$ only when $\mathbf{r}_m^e$ coincide with nodal points $\mathbf{r}_i$. In this way the final M_{ij}^{lift} becomes very sparse

$$M_{ij}^{\text{lift}} = (M)_{ik}^{-1} M_{kj} = \begin{cases} J_m^e w_m^e, & \mathbf{r}_i = \mathbf{r}_j = \mathbf{r}_m^e \\ 0, & \text{otherwise} \end{cases} \quad (26)$$

It should be noted that if $\mathbf{r}$ is on the edge, $\ell_i(\mathbf{r})$ is zero for all nodal points i that do not reside on edge, meaning that we only need those g_j that are on that edge to represent the boundary flux. Therefore the original edge mass matrix M_{ij}^e is actually of size $N_p \times N_{\text{fp}}$ [9] where N_{fp} is number of nodal points on one edge of element.

The sparse pattern of simplified M_{ij}^{lift} is given in Fig. 5. Compared with its baseline full matrix version shown in Fig. 4, only point-wise multiplication is needed to evaluate each non-zeros of boundary flux contribution $B_{a,i}$, while for baseline formulation, dot product with a length of N_{fp} is required. Significant amount of computation costs can be saved in this way.

2.2.4. Limitations and comparison with other formulations

In the present section we summarize the approximation made in this formulation and point out its limitations. Comparison with other formulations is also presented.

First, to derive the baseline formulation, $\mathbf{n} \cdot (\mathbf{F} - \mathbf{F}^*)$ need to be expressed by N -th-order Lagrangian polynomials. The advection flux $\mathbf{F}$ is already the linear function of $\tilde{f}_a$ of order N , so we only require the edge normal vector $\mathbf{n}$ to be constant along an edge in order to achieve exact representation. All triangles, tetrahedrons and quadrangles with straight edges fulfill this requirement. Arbitrary hexahedrons may have warped faces which violate this rule. But it is still a reasonable approximation as most mesh generators tend to create elements with minimal face warpage.

In section 2.2.2, to utilize the sparse differential matrices in final computation, derivative of Lagrangian polynomial basis $\frac{\partial \ell_j(\mathbf{r})}{\partial \mathbf{r}} \frac{\partial \mathbf{r}}{\partial \mathbf{x}}$ also needs to be expressed by N -th-order Lagrangian polynomials. Since $\frac{\partial \ell_j(\mathbf{r})}{\partial \mathbf{r}}$ is exactly $(N-1)$ -th-order polynomials, the coordinate mapping

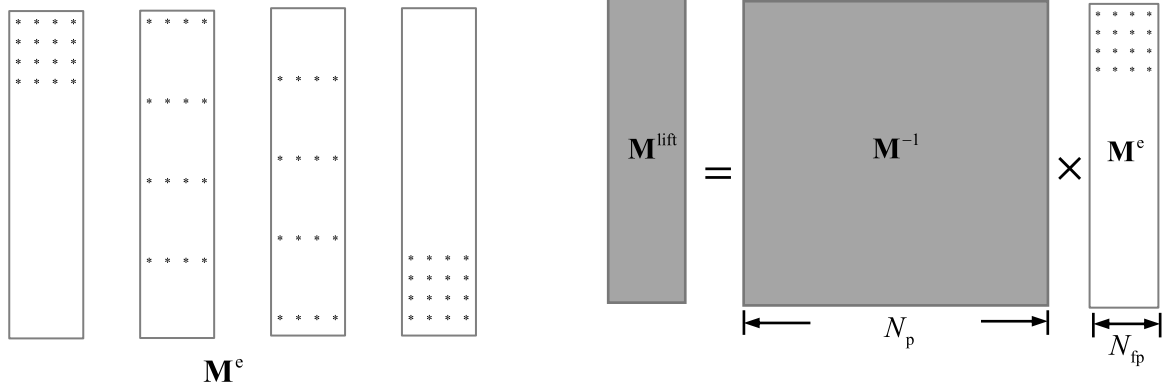


Fig. 4. The sparse pattern of general M_{ij}^c (left) and M_{ij}^{lift} (right) in quadrilateral elements. $N = 3$, $N_p = (N + 1)^2$ and $N_{fp} = N + 1$. The dark region and the asterisks represent non-zeros.

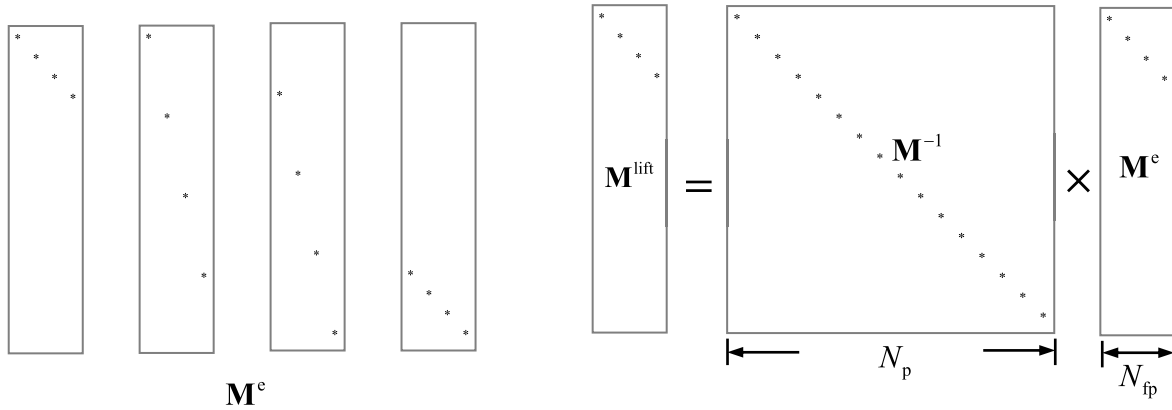


Fig. 5. The sparse pattern of simplified M_{ij}^c (left) and M_{ij}^{lift} (right) in quadrilateral elements when nodal points coincide with quadrature points.

derivate $\frac{\partial r}{\partial x}$ should be either constant (affine mapping) or linear function (bi-linear or tri-linear mapping), which covers most common element types.

In section 2.2.3 we choose Gauss-Lobatto quadrature to construct diagonal matrices but its accuracy is insufficient because the integrand is at least $2N$ -th-order while N -th-order Gauss-Lobatto quadrature can only reach the $(2N - 1)$ -th order of accuracy. Previous literature deems it acceptable [19] and we will show in section 4 that such approximation does not affect the high-order spatial convergence of DGLBM.

Due to the above approximations, we do not recommend to use this simplified formulation with highly curved elements. Besides, section 2.2.2 and 2.2.3 also require the element to be either quadrilateral or hexahedral.

The derivation in section 2.2.2 is inspired by the nodal discontinuous Galerkin method of Hesthaven and Warburton [9]. However, they use simplex elements whose differentiation matrix does not have good sparse properties to reduce computation. Besides, the mass matrices in their method cannot be sparse, either. Theoretically speaking, the present method is superior in terms of computing performance.

The true competitor is the matrix-free DG method implemented in the deal.II [24]. It is also designed for tensor-product spectral elements but heavily depends on the sum-factorization technique to reduce computation in matrix multiplication. For example, the evaluation of $D_{r,ij} f_j$ in the present formulation requires $(N + 1)^{d+1}$ fused multiply-add operations due to sparse property. By using sum-factorization technique, the same task requires $d(N + 1)^{d+1}$ operations. The computing complexity is similar because we all utilize tensor-product properties but in different ways. The matrix-free DG infrastructure in deal.II provides a very efficient way to handle general cases such as over-integration, but our

method is specially optimized for solving linear advection equations in DGLBM.

3. Implementation

3.1. Key points of implementation

3.1.1. Evaluate right hand side and perform stage

Overall, we avoid the use of global system matrix and evaluate the right hand side of Eq. (16) by one loop over elements as shown in Algorithm 1. First, we loop edges of one element and gather the solution values on adjacent element edges to build boundary flux g_j and evaluate $M_{ij}^{lift} g_j$. Then we evaluate $M_{ij}^{diff} \tilde{f}_{a,j}$ which is purely local for the element. Finally, the right hand side for single element is written back to global result vector and we proceed to the next element. Although the input solution data may be accessed several times, the temporary result is written to the global result vector only once. The efficiency of this algorithm has been confirmed previously by Kronbichler and Kormann [24] and is called *element-wise face integrals*.

The low-storage RK method requires at least one additional storage of global vector df . However, this would require another loop over global vector to update solution vector f , which is unfavorable to cache performance. So we add another storage f' to hold the updated result and the solution is updated by swapping the pointer. In this way, the RK stage can be performed with only one loop.

3.1.2. Evaluate differentiation term and vectorization strategy

The M_{ij}^{diff} is unique for each element and each discrete velocity. Although after simplification, M_{ij}^{diff} contains fewer non-zeros, if all these matrices are pre-computed and stored, the memory cost will be

Algorithm 1 Evaluation of right hand side and perform stage.

Input: distribution function $f_{a,j,k}$, additional storage $f'_{a,j,k}$ and $df_{a,j,k}$.
Output: updated result vectors f and f'

- 1: for each element k in the domain
- 2: initialize local result $RHS_{a,j}$ to zeros
- 3: for each edge of element k
- 4: gather solution values $f_{a,j}^+$ on the adjacent element edge
- 5: gather solution values $f_{a,j}^-$ on the local element edge
- 6: compute boundary flux $g_{a,j} = (f_{a,j}^- - f_{a,j}^+)e_a \cdot n_j$ for only $e_a \cdot n_j < 0$
- 7: compute flux contributions on local results $RHS_{a,j} = RHS_{a,j} + M_{ij}^{diff} g_{a,j}$
- 8: end
- 9: compute differentiation term $RHS_{a,j} = RHS_{a,j} + M_{ij}^{diff} f_{a,j,k}$
- 10: update global vector and perform RK stage:
- 11: $df_{a,j,k} = a_s^{RK} df_{a,j,k} + \Delta t RHS_{a,j}$
- 12: $f'_{a,j,k} = f_{a,j,k} + b_s^{RK} df_{a,j,k}$
- 13: end
- 14: swap the pointer f and f' so that the f is updated for next stage.

Algorithm 2 Evaluate differentiation term for D3Q19 lattice.

Input: distribution function $f[\alpha, j]$, differentiation matrices $Dx[i, j]$, $Dy[i, j]$, and $Dz[i, j]$, discrete velocity $ex[\alpha]$, $ey[\alpha]$, and $ez[\alpha]$, where $\alpha = 1, 2, \dots, 18$, $i, j = 1, 2, \dots, N_p$.
 $ex = [1, -1, 0, 0, 0, 0, 1, -1, 1, -1, 1, -1, 1, -1, 0, 0, 0, 0]$;
 $ey = [0, 0, 1, -1, 0, 0, 1, -1, 1, -1, 1, -1, 1, -1, 0, 0, 0, 0]$;
 $ez = [0, 0, 0, 0, 1, -1, 0, 0, 0, 0, 1, -1, 1, -1, 1, -1, 0, 0]$.

Output: result of differentiation term $df[\alpha, j]$

Prepare temporary array $tmpx$, $tmpy$, $tmpz$ of length 8, tmp of length 6 and initialize df

- 1: for each row index i of Dx
- 2: $tmp = 0$; $tmpx = 0$; $tmpy = 0$; $tmpz = 0$;
- 3: for each column index j of non-zeros in i th row of Dx
- 4: $tmp[1 : 2] = tmp[1 : 2] + Dx[i, j] * f[1 : 2, j]$
- 5: $tmp[3 : 4] = tmp[3 : 4] + Dy[i, j] * f[3 : 4, j]$
- 6: $tmp[5 : 6] = tmp[5 : 6] + Dz[i, j] * f[5 : 6, j]$
- 7: $tmpx[1 : 8] = tmpx[1 : 8] + Dx[i, j] * f[7 : 14, j]$
- 8: $tmpy[1 : 4] = tmpy[1 : 4] + Dy[i, j] * f[7 : 10, j]$
- 9: $tmpz[1 : 4] = tmpz[1 : 4] + Dz[i, j] * f[7 : 10, j]$
- 10: $tmpz[5 : 8] = tmpz[5 : 8] + Dy[i, j] * f[15 : 18, j]$
- 11: $tmpz[1 : 8] = tmpz[1 : 8] + Dz[i, j] * f[11 : 14, j]$
- 12: $df[1 : 2] = df[1 : 2] + eax[1 : 2] * tmp[1 : 2]$
- 13: $df[3 : 4] = df[3 : 4] + eay[3 : 4] * tmp[3 : 4]$
- 14: $df[5 : 6] = df[5 : 6] + eaz[5 : 6] * tmp[5 : 6]$
- 15: $df[7 : 10, i] = df[7 : 10, i] + eax[7 : 10] * tmpx[1 : 4] + eay[7 : 10] * tmpy[1 : 4]$
- 16: $df[11 : 14, i] = df[11 : 14, i] + eax[11 : 14] * tmpx[5 : 8] + eaz[11 : 14] * tmpz[1 : 4]$
- 17: $df[15 : 18, i] = df[15 : 18, i] + eay[15 : 18] * tmpy[5 : 8] + eaz[15 : 18] * tmpz[5 : 8]$
- 18: end

$qN_{cell}N_p(dN + 1)$ floating-point numbers, Here q is the number of discrete velocities. N_{cell} is the number of cells. The memory cost can be huge if N_{cell} is large, and the performance is likely to be limited by memory bandwidth. Therefore, it is necessary to make compromise between computation and memory cost. A wise trade-off is to store only the coordinate projections $D_{x,ij}$ and $D_{y,ij}$, and evaluate M_{ij}^{diff} during runtime. In this way, we only have to store $dN_{cell}N_p(dN + 1)$ floating-point numbers and dimension d is much smaller than q . Although this trade-off seems to increase the computation cost by about d times, it can be further simplified, as many zeros exist in the discrete velocity e_α (see Algorithm 2).

The codes can be efficiently vectorized with careful design, as shown in Algorithm 2. We prefer vectorization over α because they share the same set of metrics. By comparison, in deal.II, a few elements are grouped for vectorization, which is more general to handle both vector-valued and scalar problems.

3.1.3. Parallel implementation

The DGLBM program is written in Fortran and follows the latest Fortran 2018 standard. The parallel implementation includes the OpenMP version for shared-memory platforms and Coarray Fortran (CAF) [27] version for distributed memory platforms. The coarray features in Fortran 2008 and later provide a means of distributed parallel programming without touching the complicated MPI interfaces, which is efficient for building prototypes. To assist parallel processing, METIS [26] software can efficiently partition large unstructured meshes and reorder the sequence of elements. For the OpenMP version, we utilize the thread

Table 1

Detailed setup for performance comparisons.

CPU	Intel Xeon Gold 5118, 2.29 GHz, 12 cores
Memory	6-channel DDR4 2400 MHz
Parallel	8 cores in 1 CPU package, OpenMP
Compiler	Intel Fortran Compiler for Linux 19.1, -O3 -xCORE-AVX512 -ipo
Problem size	32^3 hexahedrons / $32^3 \times 6$ tetrahedrons
Algorithm	D3Q19, BGK, 5-stage low-storage RK

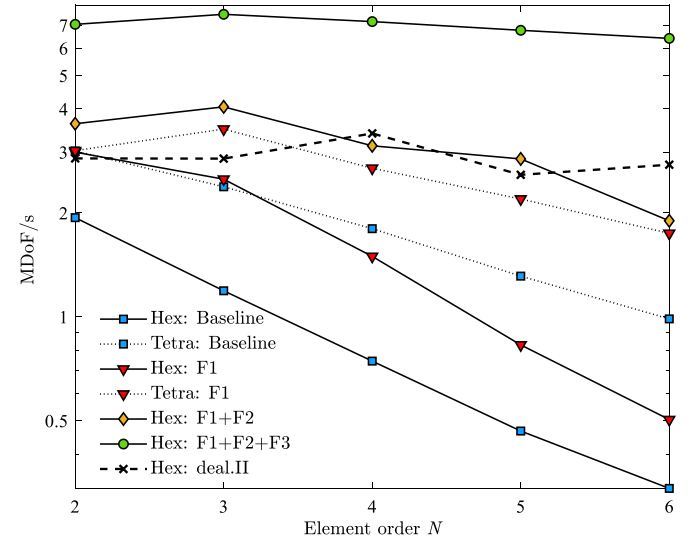


Fig. 6. Performance comparison of different type of elements and Algorithms. The performance is measured in MDof/s for streaming process only. F1 + F2 + F3 refers to the combination of all three simplified formulations and achieves the best result.

affinity and *First-Touch* policy to ensure the locality of data placement, which improves the scaling performance on NUMA systems [28].

3.2. Performance tests and discussion**3.2.1. Performance comparison of different formulations**

First, different variants of nodal DG formulation are tested and compared. Detailed setup is summarized in Table 1. The computing speed can be measured by million degree of freedoms updated per second (MDof/s):

$$\text{MDof/s} = \frac{N_{\text{Tstep}} N_{\text{cell}} N_p}{t_{\text{run}}} \times 10^{-6} \quad (27)$$

where N_{Tstep} is the number of time steps and t_{run} is the total run time.

The widely-used element types and different formulation variants are tested. The three simplified formulations in section 2.2.1~2.2.3 are named F1 ~ F3, respectively, and only F1 can be applied to tetrahedrons. The baseline refers to the general full matrix version of Eq. (11). Note that when the same domain is discretized with tetrahedrons, each hexahedron is filled by six tetrahedrons to reach a similar grid resolution. We also implement the D3Q19 streaming step using the matrix-free facility of deal.II, primarily based on the tutorial code step-67 and step-76 [25]. As is shown in Fig. 6, the combination of simplified formulations achieves the best performance among all variants. Moreover, its performance varies little for tested element order (2~6), making higher-order discretization a more competitive choice in practical simulations.

We also test the performance of Palabos [29], a well-known standard LBM library on the same hardware. The test grid for Palabos is 96^3 , corresponding to the 3rd order 32^3 mesh in DGLBM. The performance for LBM is usually measured by the number of lattices updated per second (MLUPS), equivalent to MDof/s in DGLBM. The test shows DGLBM has an overall speed of about 6.5 MDof/s while Palabos reaches around

Table 2
Detailed setup for scaling tests.

	Strong scaling test	Weak scaling test
Machine	Workstation, 4-socket Intel Xeon Gold 5118, 4 × 12 cores in total	Cluster, 2-socket AMD EPYC 7542, 2 × 32 cores per node
Memory	6-channel DDR4 2400 MHz	8-channel DDR4 3200 MHz
Compiler	Intel Fortran Compiler for Linux 19.1, -O3 -xCORE-AVX512 -ipo	Intel Fortran Compiler for Linux 19.1, -O3 -mavx -ipo
Problem size	4 th order 60 ³ hexahedral elements	3 rd order 20 ³ hexahedral elements per core
Algorithms	D3Q19, BGK, 5-stage 2-order low-storage RK, F1 + F2 + F3	

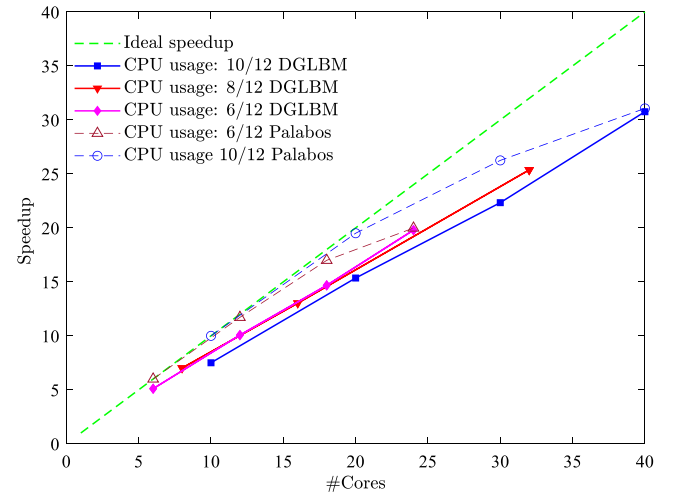
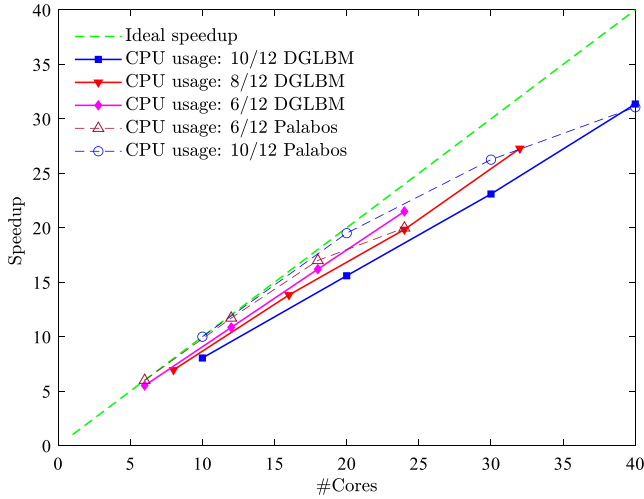


Fig. 7. Speedup of OpenMP (left) and CAF (right) DGLBM programs in strong scaling tests on a 4-socket workstation. The problem size is 4th order 60³ hexahedral elements. The speedup is measured with respect to single core. CPU usage of 10/12 indicates the 10 out of 12 cores in one CPU package are utilized for parallel computing.

53 MLUPS on the same hardware resource. Our DGLBM can reach 1/9 the speed of Palabos. This result is superior to the NATrium library by Kramer et al. [18]. Their test shows the SLLBM can reach about 10 MLUPS while Palabos can reach 140 MLUPS, that is, the SLLBM can reach 1/14 the speed of Palabos. The SEDG-LBM in NATrium, equivalent to DGLBM here, is even slower with a speed less than 2 MLUPS. Note that we use five-stage RK method for time marching in DGLBM while for SLLBM in NATrium, time marching is completed by only one sparse matrix-vector multiplication, which is given as [18]

$$f_{a,ci}(t) = \sum_{j=1}^{N_p} f_{a,\zeta j}(t - \Delta t) \psi_{\zeta j}(\mathbf{x}_{ci} - \Delta t \mathbf{e}_a), \quad (28)$$

where ci refers to the support points of the target element and ζj refers to the support points of the source element where $\mathbf{x}_{ci} - \Delta t \mathbf{e}_a$ locates. $\psi_{\zeta j}(\mathbf{x})$ is the shape function that interpolates function values of ζj points to arbitrary location $\mathbf{x}$. As is shown in Eq. (28), it requires N_p multiply-add operations to update one support point and $N_p^2 = (N+1)^{2d}$ operations to update the whole element, which costs more than the simplified DG formulation. However, it is also possible to implement sum-factorization and matrix-free method in SLLBM to gain more performance and we plan to work on it in the future.

3.2.2. Scaling performance

For completeness, we also present parallel scaling tests for both OpenMP and CAF version of DGLBM programs. Since we have not specially optimized our codes for scaling efficiency, the tests results may not be optimal. We plan to improve it in the future release. The detailed setup is given in Table 2.

The speedup in strong scaling is measured with respect to the single-core performance (see Fig. 7). If we modify the binding strategy to maintain constant CPU usage, we find the parallel efficiency varies little and the linearity is good. The increase of CPU usage causes a significant

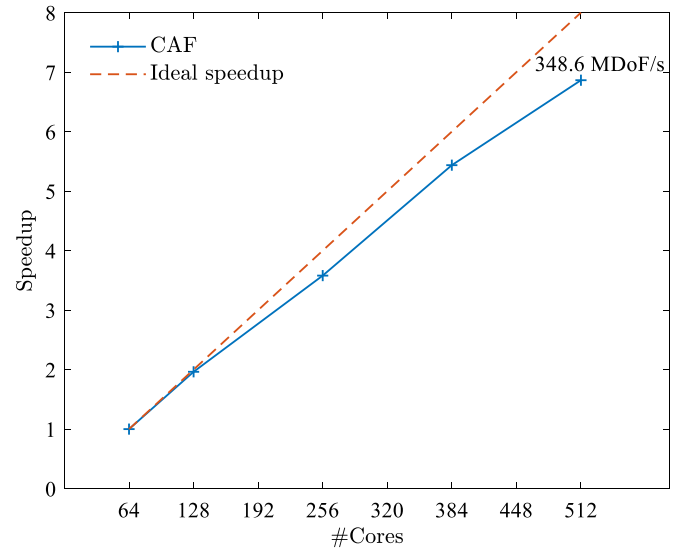


Fig. 8. Speedup of CAF program in weak scaling tests on clusters. The task load for each core is 20³ 3rd order hexahedral elements. The speedup is measured with respect to single node. Each node contains 2-socket CPU and 64 cores in total.

drop in parallel efficiency, implying obvious memory-bound effect. The CAF version program, which has more overhead due to inter-process communication, reaches about 80% performance of the OpenMP version but shares similar parallel efficiency. The Palabos library shows different scaling performance. For medium and small parallel scale, the parallel efficiency is very high, while for larger parallel scale with over 20 cores, the efficiency drops significantly due to the memory bound.

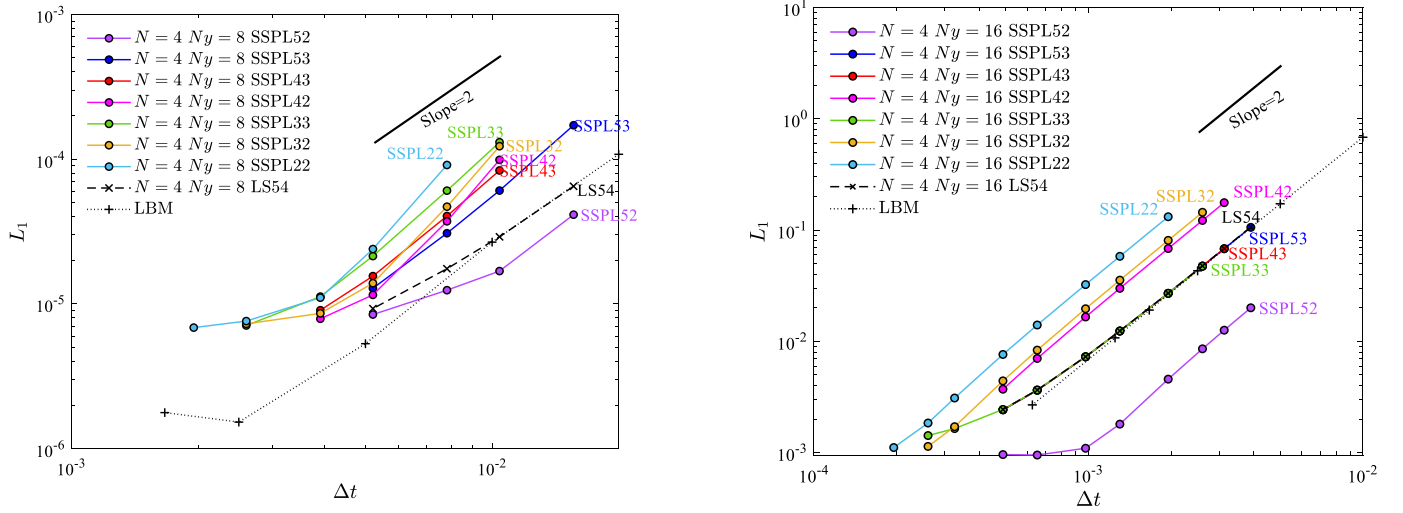


Fig. 9. The convergence of error norm with various time step size Δt and different RK methods. Left: 2-D Taylor-Green vortex with $Re = 10$, $Ma = 0.01$. Right: 1-D standing wave.

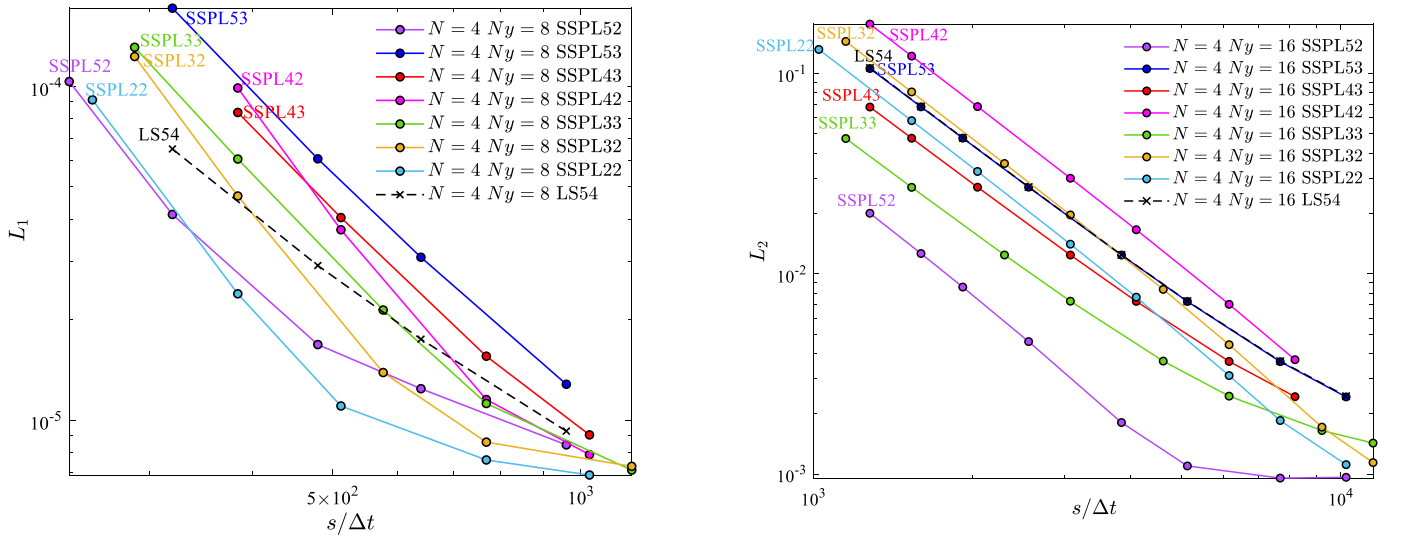


Fig. 10. The error norm versus relative computing cost. Left: 2-D Taylor-Green vortex with $Re = 10$, $Ma = 0.01$. Right: 1-D standing wave.

With 40 cores in parallel computing, the speedup of Palabos become close to DGLBM.

The speedup in weak scaling tests, measured with respect to single node, is shown in Fig. 8 and we see a reasonable weak scaling efficiency for 8 nodes, 512 cores.

4. Numerical tests

4.1. Validation and convergence study

4.1.1. Cases description

We mainly focus on unsteady flow simulation because this is the inherent capability of DGLBM. Two simple benchmark cases, the two-dimensional Taylor-Green vortex and the one-dimensional standing wave are chosen in this convergence study. The former belongs to incompressible flow while the latter belongs to acoustics.

The two-dimensional Taylor-Green vortex is defined on periodic square domain $[-\pi L, \pi L]^2$ with the following incompressible analytical solution:

$$\begin{aligned} u_x^{\text{ref}}(\mathbf{x}, t) &= V_0 \sin\left(\frac{x}{L}\right) \cos\left(\frac{y}{L}\right) \exp\left(-2\frac{\nu}{L^2}t\right) \\ u_y^{\text{ref}}(\mathbf{x}, t) &= -V_0 \cos\left(\frac{x}{L}\right) \sin\left(\frac{y}{L}\right) \exp\left(-2\frac{\nu}{L^2}t\right) \\ p^{\text{ref}}(\mathbf{x}, t) &= p_0 + \frac{1}{4}\rho_0 V_0^2 \left[\cos\left(\frac{2x}{L}\right) + \cos\left(\frac{2y}{L}\right) \right] \exp\left(-4\frac{\nu}{L^2}t\right), \end{aligned} \quad (29)$$

where V_0 , L , ρ_0 , p_0 are all constants and $p_0 = \rho_0 c_s^2$. Reynolds number $Re = V_0 L / \nu$ and Mach number $Ma = V_0 / c_s$ is sufficient to define the flow parameters. Equilibrium state is chosen as the initial field of distribution function. To account for both temporal error and spatial error, the error norm is defined as the average of all spatial and temporal samples:

$$L_1 = \frac{1}{M_i M_j} \sum_{j=1}^{M_j} \sum_{i=1}^{M_i} |u(\mathbf{x}_i, t_j) - u^{\text{ref}}(\mathbf{x}_i, t_j)|, \quad (30)$$

where M_i and M_j are numbers of the sampling points $\mathbf{x}_i$ and t_j , respectively.

The one-dimensional standing wave is defined between two straight walls at $y = 0$ and $y = L$. For compatibility it is still computed in two-dimensional cases with D2Q9 lattice. Periodic boundary is set in x

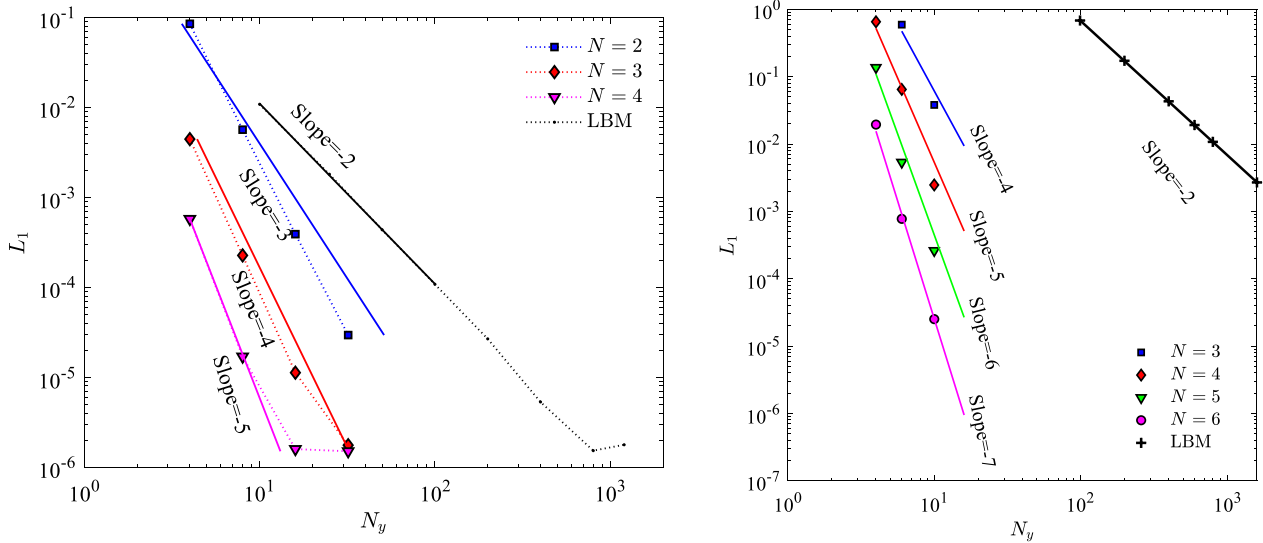


Fig. 11. The error norm obtained from $\Delta t \rightarrow 0$ with different grid resolution N_y and element order N . Left: 2-D Taylor-Green vortex with $Re = 10$, $Ma = 0.01$. Right: 1-D standing wave case. Lower error bound of magnitude 10^{-6} can be observed in 2-D Taylor-Green vortex, corresponding to the modeling error due to finite Mach number.

direction. Linear LB model [4] with zero viscosity is set for this simulation and the analytical solution is

$$\rho^{\text{ref}}(y, t) = \rho_0 + \Delta\rho_0 \cos\left(8\pi \frac{y}{L}\right) \cos\left(8\pi \frac{c_s t}{L}\right), \quad (31)$$

where $\Delta\rho_0$ is the amplitude of standing wave and the wave length is $\frac{L}{4}$. The error norm is defined similar to (30) with velocity u replaced by density ρ .

4.1.2. Results and discussion

For both cases, the error norms L_{err} with respect to analytical solution can be divided into three parts, temporal error $L_{\Delta t}$, spatial error $L_{\Delta y}$ and modeling error L_{model} . Therefore, our numerical tests start from a relatively large Δt and the convergence of time error $L_{\Delta t}$ can be seen in Fig. 9. As Δt is close to zero and $L_{\Delta t}$ become negligible, the spatial error $L_{\Delta y}$ is exposed. The convergence of $L_{\Delta y}$ with relation to the number of elements along y -axis N_y and element order N is given in Fig. 11. Finally, when temporal and spatial discretization are both fine enough, the remaining error should be the modeling error L_{model} , usually related to flow speed and shown in Fig. 12. The results of the standard LBM are also plotted for comparison.

We first examine the convergence of temporal error with different RK methods. From Fig. 9, all methods lead to an approximately second-order convergence rate. As DGLBM is based on LBE which adopts second-order time integration, we do not expect it to achieve a higher-order accuracy in time. However, the absolute error produced by different RK methods still varies a lot in different cases. Overall, RK methods with more stages tend to produce higher accuracy. An interesting finding is that SSPL52 can produce much lower errors than the standard LBM in the standing wave case, which indicates the superiority of SSPL52 for acoustic simulations. Since the computing cost is proportional to the stage number s and the number of time steps, $s/\Delta t$ can be used to measure the relative computing cost of different RK methods. As is shown in Fig. 10, for 1-D standing wave, SSPL52 is undoubtedly the most efficient method while for 2-D Taylor Green vortex, SSPL22 seems better in general. SSPL52 becomes as efficient as SSPL22 only with larger time step sizes. However, since SSPL52 is more stable and gives better results under large time step sizes, we still use SSPL52 in the following turbulence cases.

CFL number is often used to measure the relative size of time step which is useful for both stability and accuracy. An approximate but practicable CFL formula is given as follows:

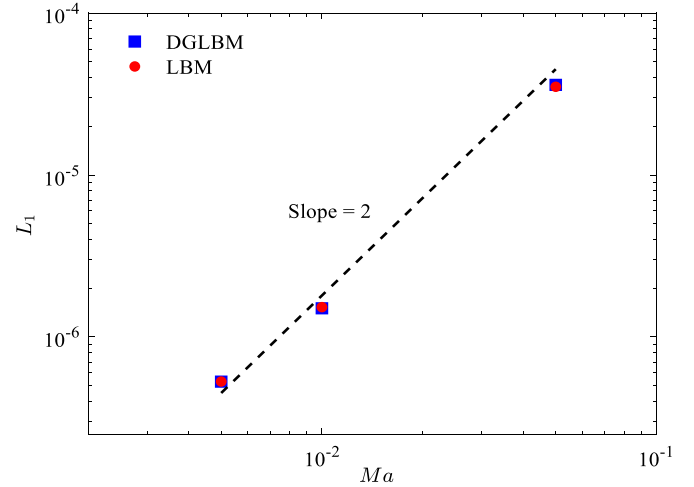


Fig. 12. The relation between modeling error and the Mach number in 2-D Taylor-Green vortex. The modeling error is obtained when temporal and spatial discretization are both fine enough.

$$CFL = \frac{|e_a| \Delta t}{\min(h)/N^2}, \quad (32)$$

where h is the characteristic length of an element, for example, the diameter of inscribed sphere. The Taylor-Green vortex can easily reach time convergence with relatively large CFL number while the standing wave case needs much smaller CFL number to eliminate the temporal error. This is because the density fluctuations are transmitted in sound speed which is much faster than the vortex evolution. Thus higher temporal resolution is needed to capture the sound waves accurately.

Fig. 11 shows that the spatial error converges with an approximate order of $N + 1$, which is a common property of high-order discontinuous Galerkin method [9]. Krämer et al. [18] obtained N -th-order of spatial convergence possibly because they kept constant time step size during convergence test which could not eliminate the impact of temporal error.

The Taylor-Green vortex cases also exhibit modeling error with quadratic relation to Ma as is shown in Fig. 12, which corresponds to the well-known compressibility error of LBM. Fig. 12 also shows the modeling error of DGLBM has no difference compared with standard LBM.

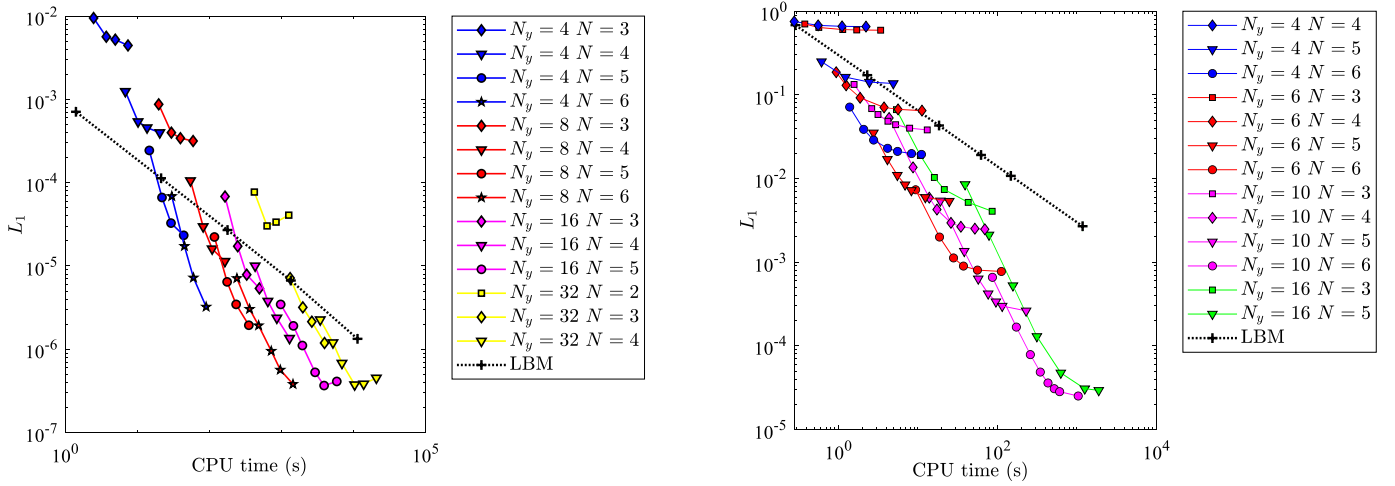


Fig. 13. The error norm versus CPU time. Left: 2-D Taylor-Green vortex with $Re = 10$, $Ma = 0.005$. Right: 1-D standing wave. For DGLBM, each line consists of cases with the same spatial discretization setup but different time step sizes. The computing cost of LBM is measured from Palabos library.

Computing efficiency is a very interesting topic. Fig. 13 shows the error norm versus CPU time for different case setups. Each line in the figure consists of cases with different time step size. It is clearly shown that high-order discretization can achieve better accuracy with less computing cost. For the Taylor-Green vortex, DGLBM becomes more efficient than LBM when accuracy is high enough, while for the standing wave, DGLBM is superior to LBM in most cases.

4.2. The effect of h/p refinement on DNS of turbulent flow

4.2.1. Case description

The Taylor-Green vortex in a three-dimensional tri-periodic cubic box is a well-known turbulent flow case that covers many phenomena including the stretch and breakdown of vortices, the transition and free decaying of turbulence. It starts from a simple smooth solution, making it an ideal turbulence benchmark case [30]. Here we take the pseudo-spectral solution given in [31] for reference and follow its case setup.

An initial field in the following form is defined in the tri-periodic box $\Omega = [-\pi L, \pi L]^3$.

$$\begin{aligned} u_x^{\text{ref}}(\mathbf{x}, 0) &= V_0 \sin\left(\frac{x}{L}\right) \cos\left(\frac{y}{L}\right) \cos\left(\frac{z}{L}\right) \\ u_y^{\text{ref}}(\mathbf{x}, 0) &= -V_0 \cos\left(\frac{x}{L}\right) \sin\left(\frac{y}{L}\right) \cos\left(\frac{z}{L}\right) \\ u_z^{\text{ref}}(\mathbf{x}, 0) &= 0 \end{aligned} \quad (33)$$

$$p^{\text{ref}}(\mathbf{x}, 0) = p_0 + \frac{1}{16} \rho_0 U_0^2 \left[\cos\left(\frac{2x}{L}\right) + \cos\left(\frac{2y}{L}\right) \right] \left[\cos\left(\frac{2z}{L}\right) + 2 \right],$$

where V_0 , p_0 , ρ_0 are constants. The Reynolds number is $Re = V_0 L / \nu = 1600$ and Mach number is $Ma = V_0 / c_s = 0.1$. The average energy E_k , and enstrophy ξ are defined as follows.

$$E_k(t) = \frac{1}{(2\pi L)^3} \int_{\Omega} \frac{\mathbf{u} \cdot \mathbf{u}}{2} d\Omega, \quad \xi(t) = \frac{1}{(2\pi L)^3} \int_{\Omega} \frac{\boldsymbol{\omega} \cdot \boldsymbol{\omega}}{2} d\Omega. \quad (34)$$

To evaluate the accuracy of simulations, it is common to monitor the evolution of dissipation rate, $-\frac{dE_k}{dt}$, also known as the measured dissipation rate. For incompressible flow, the theoretical dissipation rate is also related to enstrophy as $2\nu\xi$ and the discrepancy between $-\frac{dE_k}{dt}$ and $2\nu\xi$ can serve as an indication of numerical dissipation [33].

4.2.2. Results and discussion

Uniform meshes of three different sizes, 32^3 , 60^3 and 96^3 , together with different orders are chosen in the present simulations. Previous study has shown that the case of $Re = 1600$ converges with 512^3 grid resolution in the pseudo-spectral method [32]. In the present work,

much lower resolutions are used to show the convergence. The time step size is determined by a moderate CFL number of about 0.8, which is a balance for accuracy and efficiency. The same case is also simulated by standard LBM via Palabos library for comparison. Both LBM and DGLBM cases use D3Q19 lattice and BGK collision model. As shown in Fig. 14, $-\frac{dE_k}{dt}$ and ξ grow rapidly since $t \approx 4$ and reach the peaks at about $t \approx 9$, which corresponds to the phase of vortex breaking down and transition to turbulence. We mainly focus on the result in the range of $t < 10$ which covers the key phenomena of Taylor-Green vortex. In low-resolution cases, the ξ is significantly under-estimated near the peak but the $-\frac{dE_k}{dt}$ seems less affected, implying strong numerical dissipation.

The accuracy of Taylor-Green vortex case strongly depends on well resolving the small-scale vortices. We present the three-dimensional energy spectra compared with the reference solution in Fig. 15, calculated in the same way as [34]. Since the high-order expansion can provide solution values at any location, the input samples for fast Fourier transform do not have to be limited by grid resolution. Here the maximum wavenumber for sampling is close to the reference solution. We can see from Fig. 15 that the energy in the high wavenumber region is damped for low-resolution cases because the small-scale vortex is not well resolved. The vertical line indicates the maximum resolved wavenumber. On the left of vertical line, the relative error between the simulation result and reference is lower than 5%. By increasing the element order or refining the mesh, the gap is narrowed and the maximum resolved wavenumber increases.

To quantitatively measure the deviation from reference, the following error norms are defined similar to the previous studies [33]:

$$\Delta\epsilon_1 = \frac{\max |2\nu\xi - 2\nu\xi^{\text{ref}}|}{\max |2\nu\xi^{\text{ref}}|}, \quad \Delta\epsilon_2 = \frac{\max \left| \frac{dE_k}{dt} - \left(\frac{dE_k}{dt} \right)^{\text{ref}} \right|}{\max |2\nu\xi^{\text{ref}}|}. \quad (35)$$

Table 3 summarize the error and CPU time cost of different cases. To reach higher accuracy ($\Delta\epsilon_1 \sim 10^{-2}$), the 96^3 mesh requires at least 3rd order element while the 32^3 mesh requires at least 5th order element. Although their accuracy is around the same level, the case with 32^3 mesh requires much fewer DoFs and CPU time. Similar results can also be observed in the comparison of 96^3 mesh and 60^3 mesh, which all indicate the higher efficiency of the p -type refinement than the h -type refinement. Fig. 16 further shows the relation between error norm and CPU time, and we can find DGLBM can achieve slightly higher efficiency than LBM by using 5th order element. However, an exception is found when the order further increases to 6 in the 32^3 mesh. In this case, $2\nu\xi$ and $-\frac{dE_k}{dt}$ match well but their peak values exceed the reference solution in Fig. 14. Similar findings can also be observed in Fig. 15 where

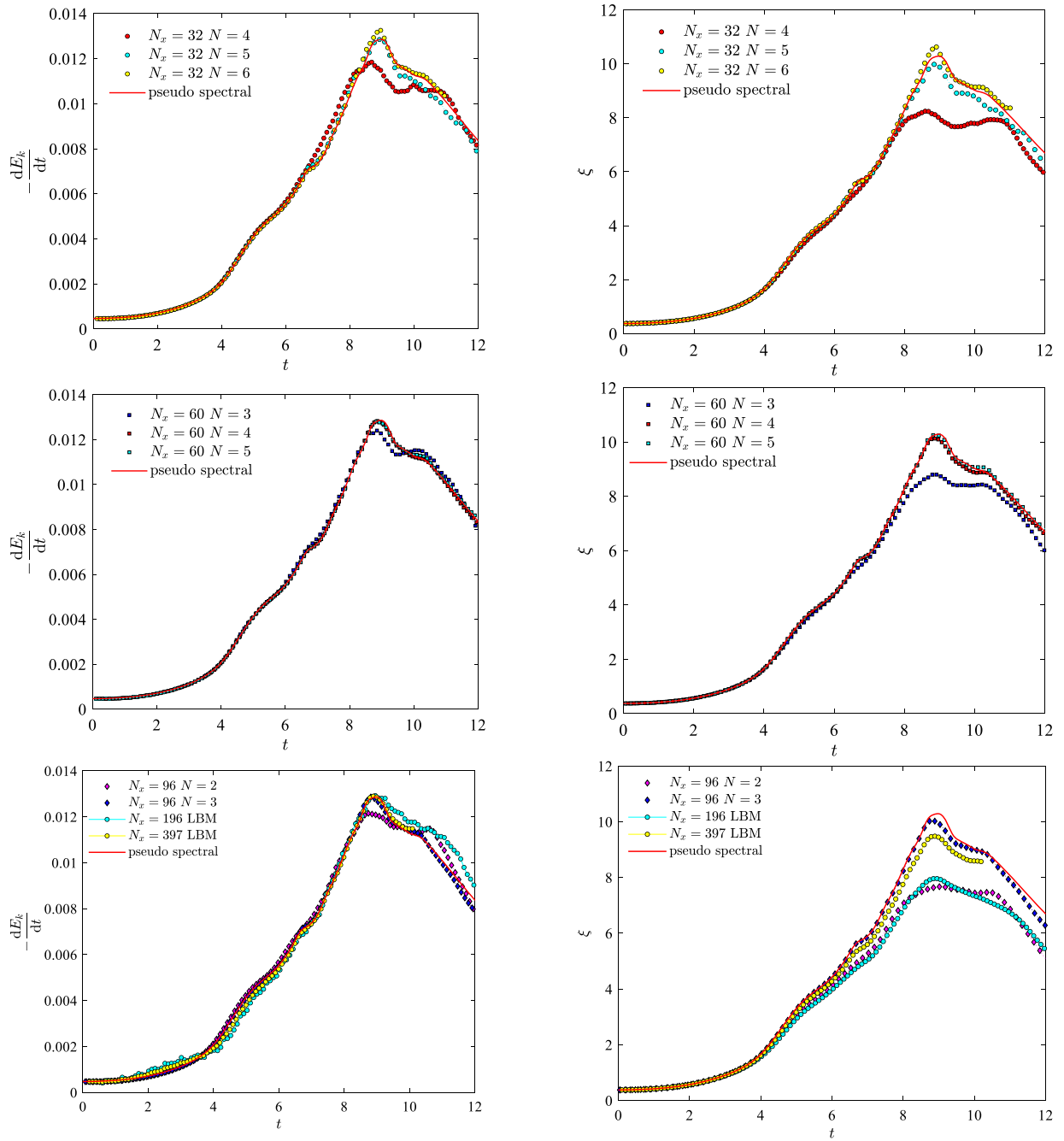


Fig. 14. Temporal evolution of measured dissipation rate $-\frac{dE_k}{dt}$ and enstrophy ξ for different discretization setups.

Table 3

Summary of discretization setups, error norms, and CPU time cost for 3-D Taylor-Green vortex.

Solver	N_x	N	DoFs	Grid resolution	$\Delta \epsilon_1$	$\Delta \epsilon_2$	CPU time /s
DGLBM	32	4	160^3	128^3	0.229	0.129	9.12×10^4
	32	5	192^3	160^3	0.0559	0.0336	2.53×10^5
	32	6	224^3	192^3	0.0364	0.0337	6.16×10^5
	60	3	240^3	180^3	0.149	0.0612	3.55×10^5
	60	4	300^3	240^3	0.0292	0.0255	1.12×10^6
	60	5	360^3	300^3	0.0085	0.0121	3.38×10^6
	96	2	288^3	192^3	0.256	0.0618	6.25×10^5
	96	3	384^3	288^3	0.0397	0.0185	2.11×10^6
Palabos (LBM)	196	-	196^3	196^3	0.227	0.0504	6.72×10^3
	397	-	397^3	397^3	0.0846	0.0250	1.16×10^5
	510	-	510^3	510^3	0.0557	0.0192	3.17×10^5
	617	-	617^3	617^3	0.0396	0.0160	6.28×10^5

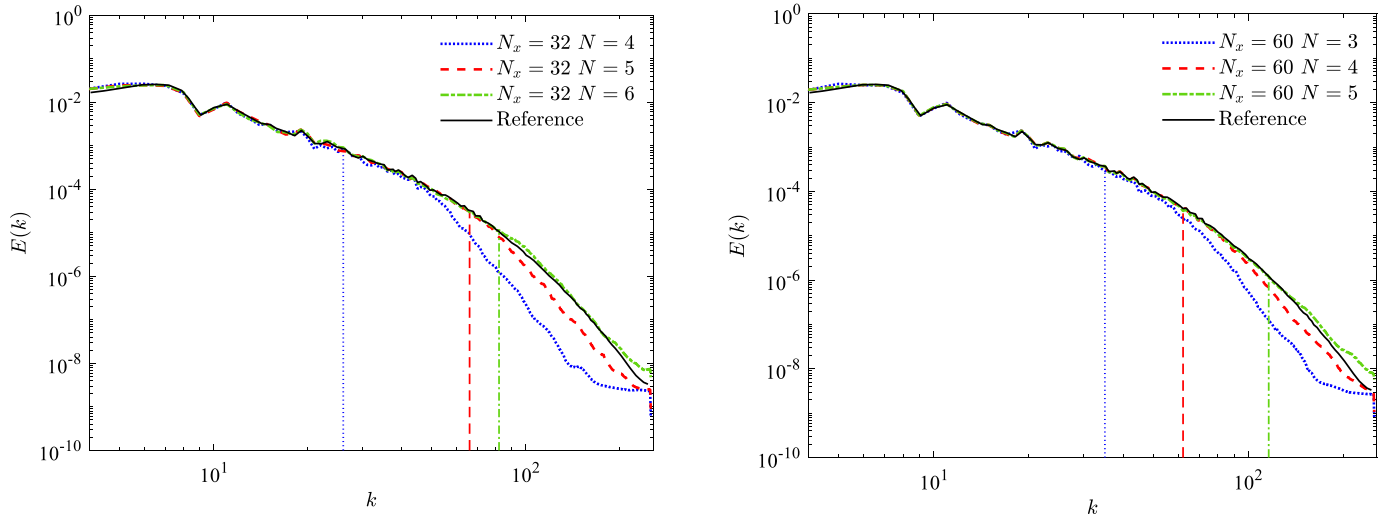


Fig. 15. Three-dimensional energy spectra at $t = 9$ in Taylor-Green vortex flow. On the left of vertical line, the relative error between the simulation result and reference is lower than 5%.

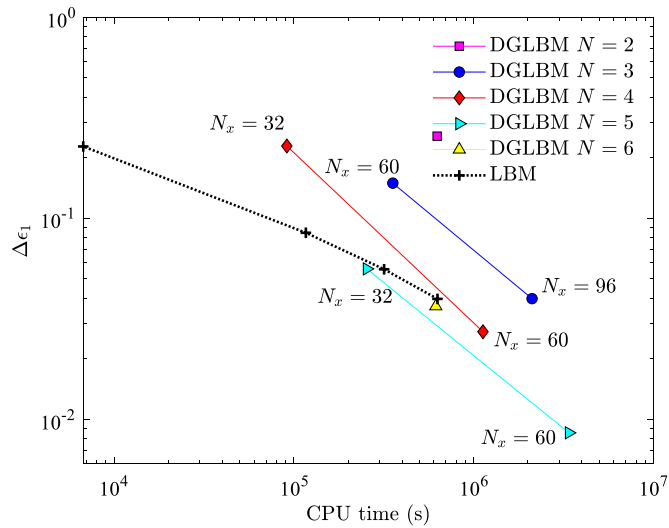


Fig. 16. The relation between error norm and CPU time in 3-D Taylor-Green vortex.

the energy spectra at high wavenumber exceed the reference value. The maximum resolved wavenumber does not improve so much, either. It is possibly due to the known issue for the high-order element that the under-resolved high-order modes tend to be overestimated [19]. Therefore, the h -type refinement is still the most reliable way to achieve high accuracy and consistent convergence, but a coarser mesh with moderate p -type refinement can be a productive choice if the efficiency is also concerned.

4.3. DNS of wall-bounded turbulent flow

4.3.1. Case description

We choose a simple low Reynolds number fully developed turbulent pipe flow case for validation. The axisymmetric curvilinear wall can be a challenge for standard LBM. Previous studies showed that the widely-used D3Q19 lattice model cannot give correct results in turbulent pipe flow [35,36]. However, the D3Q27 lattice does not encounter such a problem and is thought to have better accuracy and stability [35]. Kuwata and Suga [36] even provided theoretic evidence proving that D3Q19 and D3Q15 produce larger truncation error than D3Q27 in circular pipe flow. Only Peng et al. [37] recently reported successful

simulation with D3Q19 by carefully handling the checkerboard instability and the accuracy of curvilinear wall boundary condition. Here we show that DGLBM can also obtain accurate results with only D3Q19 lattice and BGK collision model, indicating superior accuracy and stability compared with standard LBM.

We choose a pipe length of $L = 10R$, where R is the radius of pipe. The domain is discretized by multi-block structured hexahedral mesh. The number of elements in radial (r), azimuthal (θ) and streamwise (z) directions are 32, 48, 50 and the element order is 4. The location of the first layer grid points away from the wall is $\delta^+ \approx 0.5$, where $\delta^+ = (R - r)u_\tau/\nu$ is non-dimensional distance to wall. The mesh resolution is moderate compared with previous studies [37] and once again a moderate CFL number of about 0.8 is chosen to determine the time step size. A body force of $2\rho u_\tau^2/R$ is applied in streamwise to drive the pipe flow. The Reynolds number $Re_\tau = u_\tau R/\nu \approx 180$ and the main stream Mach number $Ma = U_m/c_s = 0.1$. We use the same initial field and same form of fluctuating force as Peng et al. [37] did to trigger the development of turbulence. The pipe flow reaches the statistically steady state after about $10R/u_\tau$ similar to [37]. $6R/u_\tau$ more time is simulated to accumulate statistic averages.

4.3.2. Results and discussion

Kuwata and Suga [36] gave a theoretical analysis which illustrates that D3Q19 can result in larger error than D3Q27 in cylindrical coordinates. To confirm it in DGLBM, we compare the D3Q19 and D3Q27 results during the initial simulation period and the section profiles are shown in Fig. 17. Here, we do not add the fluctuating force, so the cross velocity should remain zero all the time. However, we do observe conspicuous cross velocity in D3Q19 case near the wall. The distribution of cross velocity has a clear relation with azimuth, corresponding to previous findings in D3Q19 LBM. We also observe non-zero cross velocity in D3Q27 but it is 6 order of magnitude lower than that of D3Q19 and we think it is most likely caused by the tiny error of mesh metrics.

However, even though D3Q19 gives larger error than D3Q27, DGLBM with D3Q19 still produces correct results without any special treatment. Fig. 18 compares streamwise mean velocity in the present work with the LBM results given by Kang and Hassan [35]. Apparently, the present DGLBM with D3Q19 lattice does not produce the unphysical velocity profile as LBM does.

We use $\langle \cdot \rangle$ to denote the averages of all time and space samples and the second-order statistics, for example, the Reynold stress $\langle u'_r u'_z \rangle$ is computed by

$$\langle u'_r u'_z \rangle = \langle u_r u_z \rangle - \langle u_r \rangle \langle u_z \rangle. \quad (36)$$

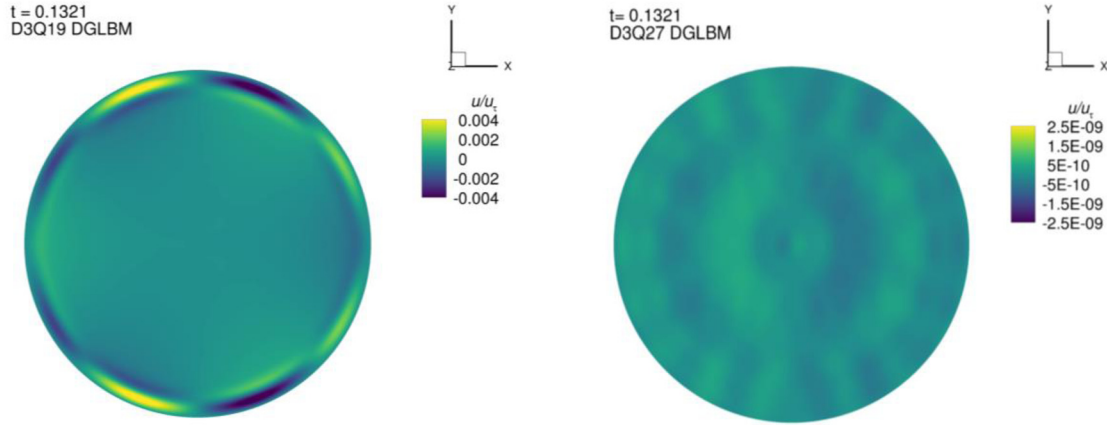


Fig. 17. Contours of x component of velocity on the pipe section. The simulation starts from a profile of zero cross velocity and no fluctuating force is added.

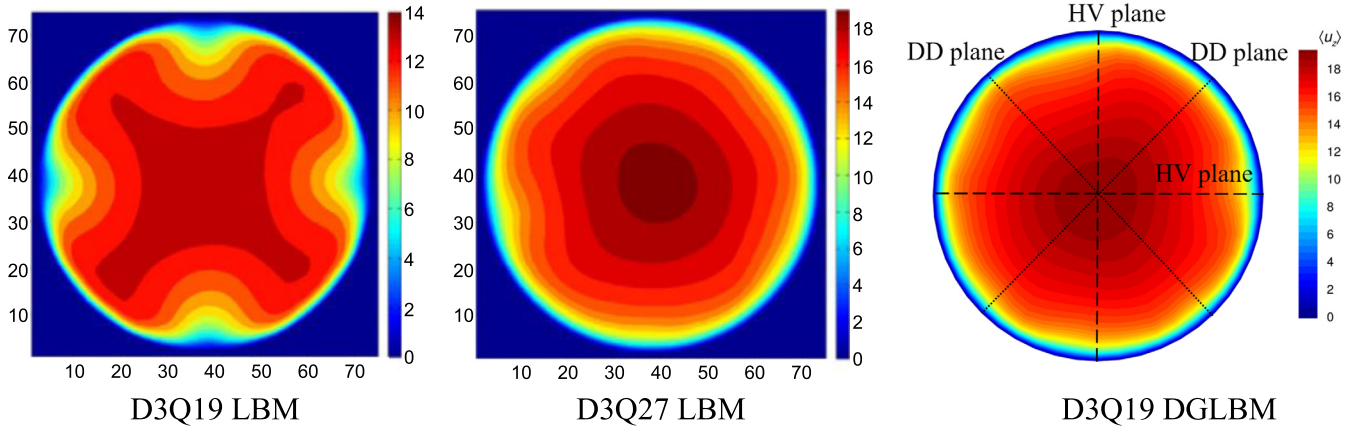


Fig. 18. Contours of streamwise mean velocity in turbulent pipe flow. The left and middle figures are given by Kang and Hassan [35] using LBM. The right figure is given by the present DGLBM.

The statistic average results are shown in Fig. 19 and compared with the reference data given by Wu and Moin [38], obtained by finite-difference method on a high-resolution grid. The LBM results given by Peng et al. [37] are also shown in the figure, which is obtained by a modified D3Q19 multi-relaxation time (MRT) model on a very dense grid. The velocity statistics, including the mean velocity and velocity fluctuation, match well the reference data and the results by Peng. The pressure statistics are more accurate than those given by Peng. The mean streamwise velocity on diagonal plane DD and axis plane HV defined in Fig. 18 is also shown in Fig. 19 (b). The discrepancy between the two results is nearly negligible, which again confirms the correctness of our simulation.

The third-order statistics skewness $S(u'_i)$ and fourth-order statistics flatness $F(u'_i)$ are also defined as follows

$$S(u'_i) = \frac{\langle u_i'^3 \rangle}{\langle u_i'^2 \rangle^{3/2}}, F(u'_i) = \frac{\langle u_i'^4 \rangle}{\langle u_i'^2 \rangle^2}. \quad (37)$$

The results of skewness and flatness are shown in Fig. 20. The high-order statistics generally match well with the reference data in the near wall region expect for the position very close to wall. The skewness of radial velocity shows an obvious jump close to the wall, which corresponds to the findings of Peng et al. [37] possibly because we do not use MRT with large bulk viscosity to dampen the acoustic waves.

Overall, DGLBM gives good results with the current D3Q19 lattice and BGK model. Besides, there is no need to impose large bulk viscosity by MRT in DGLBM for the sake of stability in simulating high-Reynolds-number flow. Large bulk viscosity may violate the physical dissipation property of acoustic waves and is not favored in acoustic simulations.

5. Conclusions

In this work, we present an efficient implementation of the nodal discontinuous Galerkin lattice Boltzmann method (DGLBM) originally given by Min and Lee [12]. The efficient implementation is based on a simplified nodal discontinuous Galerkin formulation, which makes full use of the sparse property of elemental matrices in hexahedrons and quadrangles. The simplified formulation has limited accuracy for quadrature and interpolation. Luckily, the linear advection equations in the Boltzmann method provide an ideal opportunity for this formulation. The simplified formulation saves more computing costs for high-order elements, making high-order discretization an economical choice for applications.

With careful design, the equations can be processed in vectorization efficiently and we can reach a fairly good computing performance, about the one-ninth equivalent speed of Palabos, which is even superior to the previous implementation of SLLBM or SEDG-LBM in NAtrium [18]. However, we believe there could be further improved by applying deeper optimizations on cache performance and parallel processing.

A series of numerical tests are performed which validate accuracy and convergence of our DGLBM program, followed by DNS of several benchmark turbulence cases. Among those low-storage Runge-Kutta methods we have tested, SSPL52 gives the best efficiency in acoustic cases, while SSPL22 has best efficiency in incompressible flow cases. The modeling error of DGLBM for incompressible flow shows no difference with LBM, thus many accuracy guidelines in LBM are still applicable to DGLBM. Using high-order element, DGLBM can reach bet-

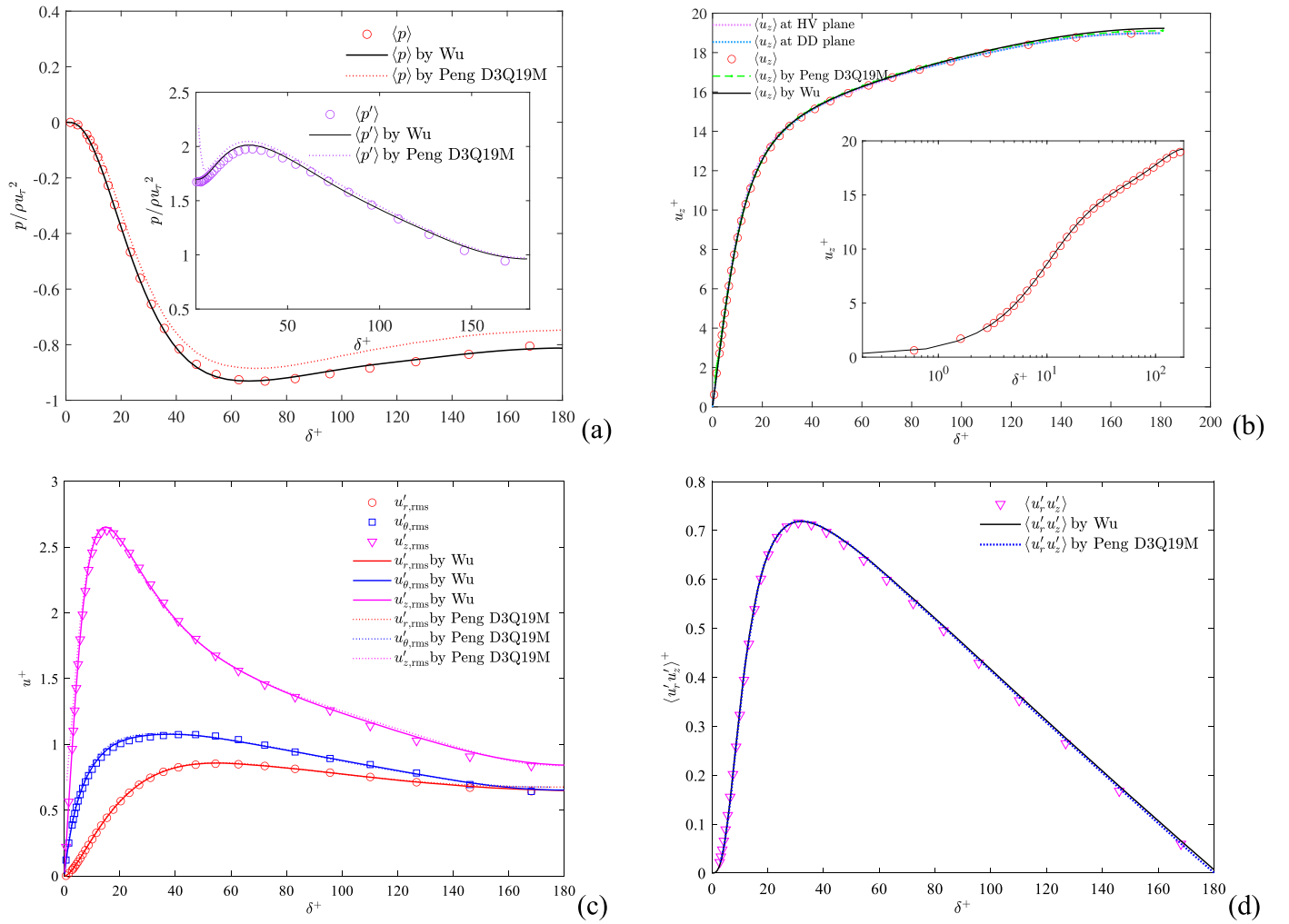


Fig. 19. Profiles of statistical average of flow quantities in turbulent pipe flow: (a) mean pressure and root mean square of fluctuating pressure, (b) streamwise mean velocity, (c) root mean square of velocity fluctuation, and (d) Reynolds stress.

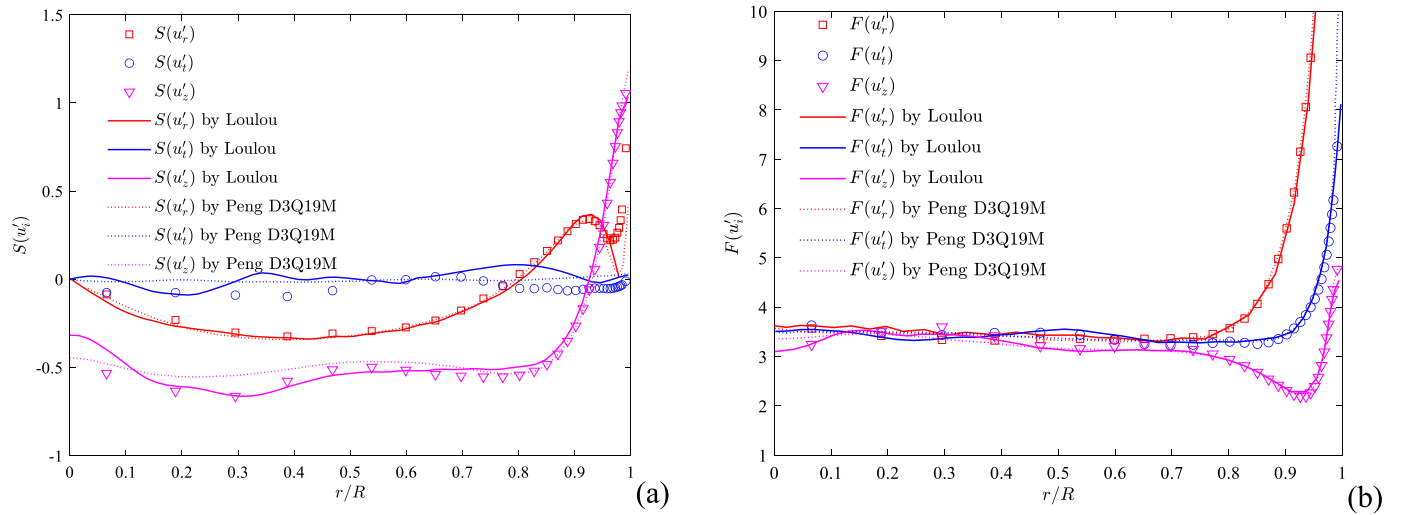


Fig. 20. Profiles of high order statistics in turbulent pipe flow: (a) skewness, (b) flatness.

ter accuracy with fewer degree of freedoms and less CPU time and even shows better efficiency than standard LBM when high accuracy is required. However, we observe an exception in under-resolved simulation of Taylor-Green vortex, showing that pure p -type refinement on

a coarse mesh may not obtain consistent convergence. Therefore, the h -type refinement is still the most reliable way to achieve high accuracy but moderate p -type refinement can effectively improve the overall efficiency in DGLBM.

The fully developed turbulent flow in circular pipe poses a challenge for standard LBM and many previous studies showed that D3Q19 lattice is inadequate to obtain correct results in this case. However, we find that DGLBM can easily obtain accurate and stable solution of turbulent pipe flow under coarse mesh with only the D3Q19 lattice and the BGK collision model.

The present study only involves the basic lattice Boltzmann model with standard lattice and BGK collision model, which can only be used for isothermal weakly compressible flow. However, the implementation techniques presented here can be easily extended to more complex Boltzmann models, giving a promising future of the higher-order discontinuous Galerkin Boltzmann method. Considering the features of DGLBM, we think there exist two promising routes for future development and application. First, it is recognized that the on-lattice standard LBM has much better performance than DGLBM and better flexibility to handle complex fluid-structure interaction. Therefore, a compromise is to develop a hybrid framework to combine the on-lattice LBM with high-order finite element based LBM. Previous studies have successfully combined FV with LBM by overlapping grids [39]. Second, The DGLBM belongs to so called off-lattice Boltzmann methods, where the discrete velocities do not have to be in lattice shape. As shown by Wilde et al. [40], without on-lattice restrictions, it may be possible to construct more ideal velocity sets that give better accuracy and stability for high-speed flow but have fewer velocities.

References

- [1] D. Kandhai, A. Koponen, A. Hoekstra, M. Kataja, J. Timonen, P.M.A. Slood, Implementation aspects of 3D lattice-BGK: boundaries, accuracy, and a new fast relaxation method, *J. Comput. Phys.* 150 (1999) 482–501.
- [2] Shin K. Kang, Yassin A. Hassan, The effect of lattice models within the lattice Boltzmann method in the simulation of wall-bounded turbulent flows, *J. Comput. Phys.* 232 (2013) 100–117.
- [3] Y. Kuwata, K. Suga, Anomaly of the lattice Boltzmann methods in three-dimensional cylindrical flows, *J. Comput. Phys.* 280 (2015) 563–569.
- [4] T. Krüger, H. Kusumaatmaja, A. Kuzmin, O. Shardt, G. Silva, E.M. Viggien, *The Lattice Boltzmann Method - Principles and Practice*, Springer Nature, Switzerland, 2017.
- [5] X. He, L.S. Luo, M. Dembo, Some progress in lattice Boltzmann method. Part I. Non-uniform mesh grids, *J. Comput. Phys.* 129 (1996) 357–363.
- [6] Z. Guo, T.S. Zhao, Explicit finite-difference lattice Boltzmann method for curvilinear coordinates, *Phys. Rev. E, Stat. Nonlinear Soft Matter Phys.* 67 (2003) 066709.
- [7] S. Ubertini, G. Bella, S. Succi, Lattice Boltzmann method on unstructured meshes, *Phys. Rev. E* 59 (1999) 4675–4682.
- [8] T. Lee, C.L. Lin, A characteristic Galerkin method for discrete Boltzmann equation, *J. Comput. Phys.* 171 (2001) 336–356.
- [9] J.S. Hesthaven, T. Warburton, *Nodal Discontinuous Galerkin Methods: Algorithms, Analysis, and Applications*, Springer Science & Business Media, New York, 2008.
- [10] X. Shi, J. Lin, Z. Yu, Discontinuous Galerkin spectral element lattice Boltzmann method on triangular element, *Int. J. Numer. Methods Fluids* 42 (2010) 1249–1261.
- [11] A. Düster, L. Demkowicz, E. Rank, High order finite elements applied to the discrete Boltzmann equation, *Int. J. Numer. Methods Eng.* 67 (2006) 1094–1121.
- [12] M. Min, T. Lee, A spectral-element discontinuous Galerkin lattice Boltzmann method for nearly incompressible flows, *J. Comput. Phys.* 230 (2011) 245–259.
- [13] Y. Li, E. Leboeuf, P.K. Basu, Least-squares finite-element scheme for the lattice Boltzmann method on an unstructured mesh, *Phys. Rev. E, Stat. Nonlinear Soft Matter Phys.* 72 (2005) 046711.
- [14] A. Karakus, N. Chalmers, J.S. Hesthaven, T. Warburton, Discontinuous Galerkin discretizations of the Boltzmann-BGK equations for nearly incompressible flows: semi-analytic time stepping and absorbing boundary layers, *J. Comput. Phys.* 390 (2019) 175–202.
- [15] A. Krämer, K. Küllmer, D. Reith, W. Joppich, H. Foysi, Semi-Lagrangian off-lattice Boltzmann method for weakly compressible flows, *Phys. Rev. E* 95 (2017) 023305.
- [16] D. Wilde, A. Krämer, D. Reith, H. Foysi, Semi-Lagrangian lattice Boltzmann method for compressible flows, *Phys. Rev. E* 101 (2020) 053306.
- [17] S. Guo, Y. Feng, J. Jacob, F. Renard, P. Sagaut, An efficient lattice Boltzmann method for compressible aerodynamics on D3Q19 lattice, *J. Comput. Phys.* 418 (2020) 109570.
- [18] A. Krämer, D. Wilde, K. Küllmer, D. Reith, H. Foysi, W. Joppich, Lattice Boltzmann simulations on irregular grids: introduction of the NATriuM library, *Comput. Math. Appl.* 79 (2020) 34–54.
- [19] G. Karniadakis, S. Sherwin, *Spectral/hp Element Methods for Computational Fluid Dynamics*, Oxford University Press, New York, 2013.
- [20] H.L. Atkins, C.W. Shu, Quadrature-free implementation of discontinuous Galerkin method for hyperbolic equations, *AIAA J.* 36 (1998) 775–782.
- [21] Z. Guo, C. Zheng, B. Shi, Discrete lattice effects on the forcing term in the lattice Boltzmann method, *Phys. Rev. E, Stat. Nonlinear Soft Matter Phys.* 65 (2002) 046308.
- [22] S.J. Ruuth, R.J. Spiteri, A new class of optimal high-order strong-stability-preserving time discretization methods, *SIAM J. Numer. Anal.* 40 (2003) 469–491.
- [23] M.H. Carpenter, C.A. Kennedy, Fourth-order 2N-storage Runge-Kutta schemes, *NASA Tech. Memo.* (1999) 109112.
- [24] M. Kronbichler, K. Kormann, Fast matrix-free evaluation of discontinuous Galerkin finite element operators, *ACM Trans. Math. Softw.* 45 (2019) 29.
- [25] Reference documentation for deal.II version 9.3.1, https://dealii.43-1.org/downloads/dealii-9.3.1-offline_documentation.tar.gz.
- [26] G. Karypis, V. Kumar, A fast and highly quality multilevel scheme for partitioning irregular graphs, *SIAM J. Sci. Comput.* 20 (1999) 359–392.
- [27] M. Metcalf, J. Reid, M. Cohen, *Modern Fortran Explained*, Oxford University Press, Oxford, 2011.
- [28] D. Schmidl, C. Terboven, OpenMP on NUMA architectures, OpenMP-Con/TWOMP, <https://iwomp2016.riken.jp/wp-content/uploads/2016/10/OpenMPonNUMAArchitectures.pdf>, 2016.
- [29] J. Latt, O. Malaspinas, D. Kontaxakis, et al., Palabos: parallel lattice Boltzmann solver, *Comput. Math. Appl.* 81 (2021) 334–350.
- [30] J.-B. Chapelier, M. de la Llave Plata, F. Renac, E. Lamballais, Evaluation of a high-order discontinuous Galerkin method for the DNS of turbulent flows, *Comput. Fluids* 95 (2014) 210–226.
- [31] Z.J. Wang, K. Fidkowski, A. Rémi, et al., High-order CFD methods: current status and perspective, *Int. J. Numer. Methods Fluids* 72 (2013) 811–845.
- [32] Wim M. van Rees, A. Leonard, D.I. Pullin, P. Koumoutsakos, A comparison of vortex and pseudo-spectral methods for the simulation of periodic vortical flows at high Reynolds numbers, *J. Comput. Phys.* 230 (2011) 2794–2805.
- [33] C. Carton de Wiart, K. Hillewaert, M. Duponcheel, G. Winckelmans, Assessment of a discontinuous Galerkin method for the simulation of vortical flows at high Reynolds number, *Int. J. Numer. Methods Fluids* 74 (2014) 469–493.
- [34] N. Sharma, T.K. Sengupta, Vorticity dynamics of the three-dimensional Taylor-Green vortex problem, *Phys. Fluids* 31 (2019) 035106.
- [35] S.K. Kang, Y.A. Hassan, The effect of lattice models within the lattice Boltzmann method in the simulation of wall-bounded turbulent flow, *J. Comput. Phys.* 232 (2013) 100–117.
- [36] Y. Kuwata, K. Suga, Anomaly of the lattice Boltzmann methods in three-dimensional cylindrical flows, *J. Comput. Phys.* 280 (2015) 563–569.
- [37] C. Peng, N. Geneva, Z. Guo, L. Wang, Direct numerical simulation of turbulent pipe flow using the lattice Boltzmann method, *J. Comput. Phys.* 357 (2017) 16–42.
- [38] X.H. Wu, P. Moin, A direct numerical simulation study on the mean velocity characteristics in turbulent pipe flow, *J. Fluid Mech.* 608 (2008) 81–112.
- [39] G. Di Ilio, D. Chiappini, S. Ubertini, G. Bella, S. Succi, Hybrid lattice Boltzmann method on overlapping grids, *Phys. Rev. E* 95 (2017) 013309.
- [40] D. Wilde, A. Krämer, M. Bedrunke, Cubature rules for weakly and fully compressible off-lattice Boltzmann methods, *J. Comput. Sci.* 51 (2021) 101355.